

SkillLink Freelance Marketplace

Frontend Documentation

A Section of the DBMS Project Report

Submitted By:

Saim, Zarish, Maryam

BSIT, 5th Semester

Database Management Systems

Academic Year 2025-2026

Chapter 1

Introduction to the Frontend

1.1 What is the Frontend?

In any web application, the work is divided between two main parts. One part runs on the server and handles things like database queries, business rules, and data processing. The other part runs in the browser and is what the user actually sees and interacts with. This second part is called the frontend.

For SkillLink, the frontend is built using React, which is a JavaScript library made by Facebook. React makes it easy to build pages that update automatically when new data arrives, without needing to reload the entire page. This is important for a platform like SkillLink where users need to see their bids, contracts, and notifications in real time without any delay.

The frontend of SkillLink does not have any database logic inside it. It does not write SQL queries. It does not connect to Oracle directly. What it does is talk to the backend through HTTP requests, and the backend takes care of all the database work. This separation of work is what makes the system clean and easy to manage.

1.2 The Role of the Frontend in SkillLink

SkillLink is a freelance marketplace where clients post projects and freelancers place bids on those projects. The frontend is responsible for giving both types of users a clear and easy interface to do their work.

From the client side, the frontend allows them to register an account, log in, post new projects, see the bids they have received, accept a bid, record a payment after the work is done, and leave a review for the freelancer. All of these actions involve sending data to the backend which then stores it in the Oracle database or reads from it.

From the freelancer side, the frontend lets them browse all open projects, place a bid on any project they are interested in, track the status of their bids, view their active

contracts, and check their earnings after a payment is recorded.

There is also a separate admin panel built into the frontend. The admin can log in through a different login page and see the full picture of the platform. The admin views are direct windows into all the data stored in the database, showing users, projects, bids, contracts, payments, reviews, disputes, and notifications all in one place.

1.3 How the Frontend Connects to the Database

Even though the frontend does not run SQL queries itself, it is still deeply connected to the database through the backend. Every time a user does something on the screen, the frontend sends a request to the backend API. The backend receives that request, runs the appropriate SQL query against the Oracle database, gets the result, and sends it back to the frontend. The frontend then shows that result on the screen.

For example, when a client opens the Projects page, the frontend sends a GET request to the backend. The backend runs a SELECT query on the projects table joined with the users and categories tables. The result comes back as a list of rows, and the frontend displays each row as a project card on the screen.

This communication happens through a tool called Axios, which is a JavaScript library that makes it easy to send HTTP requests from inside a React application. Every request that goes out includes an authorization token in the header so the backend knows who is making the request and what they are allowed to see.

1.4 Why React was Chosen

React was chosen for several reasons that are practical for a project of this type.

The first reason is that React uses something called components. A component is a small, reusable piece of the interface. For example, the sidebar navigation is one component, and it is used across all pages without writing it again. This saves time and keeps the code organized.

The second reason is that React handles state very well. State means the current condition of the data on the screen. When a user submits a bid and it is accepted, the contract page needs to update. React handles this kind of live updating in a clean way.

The third reason is that React works naturally with JSON data, which is exactly the format that the backend sends data in. When the backend returns a list of users or projects, React can take that data and display it on the screen with very little extra work.

1.5 The Full Technology Stack

SkillLink uses a three layer architecture where each layer has a clear job.

Layer	Technology	Job
Frontend	React.js	Shows the interface in the browser, sends and receives data
Backend	Node.js with Express	Receives requests, runs SQL queries, sends responses
Database	Oracle 21c XE	Stores all data in 15 tables

Table 1.1: Technology Stack of SkillLink

The frontend sits at the top of this stack. It is the only part the user ever sees. Everything else happens behind the scenes. When a user clicks a button on the frontend, a chain of events starts that goes all the way down to the Oracle database and comes back up to the screen within a fraction of a second.

1.6 Summary

The frontend of SkillLink is the face of the system. It gives clients and freelancers a place to interact with the platform, and it gives the admin a complete view of all the data. It connects to the database indirectly through the backend API using HTTP requests, and every piece of data shown on the screen comes from the Oracle database. The following chapters go through each part of the frontend in detail, showing exactly how data moves between the user and the database.

Chapter 2

Technologies Used for Data Communication

2.1 Overview

The frontend of SkillLink uses several tools and libraries to communicate with the backend and ultimately with the Oracle database. Each tool has a specific job in this communication chain. This chapter goes through each one and explains what it does and why it was used in this project.

2.2 React.js

React is the main library used to build the frontend. It was created by Facebook and is one of the most widely used frontend libraries in the world today. In the context of a database driven application like SkillLink, React plays the role of taking data that comes from the backend and turning it into something the user can see and interact with.

React works by breaking the interface into small pieces called components. Each page in SkillLink is a component, and inside each page there are smaller components like buttons, forms, and cards. When data arrives from the backend, React updates only the parts of the page that need to change, without reloading the entire page. This makes the application feel fast and responsive.

For example, when a client accepts a bid on the Bids page, React sends a PUT request to the backend, and when the response comes back confirming the contract was created, React updates the page to show the new status without the user having to refresh.

2.3 Axios

Axios is the library used to send HTTP requests from the frontend to the backend. Every time the frontend needs to read data from the database or write data to it, Axios is the

tool that carries that request to the backend.

Axios supports all the HTTP methods that are needed in SkillLink:

HTTP Method	Axios Call	Database Operation
GET	axios.get()	SELECT data from the database
POST	axios.post()	INSERT new data into the database
PUT	axios.put()	UPDATE existing data in the database
DELETE	axios.delete()	DELETE data from the database

Table 2.1: HTTP Methods and their Database Operations

Every Axios request in SkillLink is sent with an Authorization header that contains the JWT token of the logged in user. The backend reads this token to verify who the user is before running any database query. Here is how a typical Axios GET request looks in the project:

Listing 2.1: Axios GET request with Authorization header

```
1 const headers = { Authorization: 'Bearer ${token}' };
2
3 const res = await axios.get(
4   'http://localhost:5000/api/projects',
5   { headers }
6 );
7
8 setProjects(res.data);
```

In this example, the frontend sends a GET request to the backend projects route. The backend receives it, checks the token, runs a SELECT query on the projects table, and sends the rows back. The frontend then stores those rows in a state variable called projects and displays them on the screen.

2.4 React Router DOM

React Router DOM is the library that handles navigation between pages in the frontend. In a normal website, every time you go to a new page, the browser makes a request to the server and downloads a whole new HTML file. React Router DOM handles this differently. It keeps the user on the same page and just swaps out the content depending on the URL.

In SkillLink, the routes are defined in the App.js file. Here is a simplified version of how the routes are set up:

Listing 2.2: Route definitions in App.js

```
1 <Routes>
2   <Route path="/landing"      element={<Landing />} />
3   <Route path="/login"        element={<Login />} />
4   <Route path="/register"      element={<Register />} />
5   <Route path="/admin/*"       element={<AdminLayout />} />
6   <Route path="/dashboard/*"   element={<UserLayout />} />
7   <Route path="/"              element={<Navigate to="/landing" />}}
8   />
9 </Routes>
```

The dashboard routes are protected by a `PrivateRoute` component that checks whether the user is logged in before allowing access. If the user is not logged in, they are redirected to the landing page. This check is done by reading the JWT token from local storage, which was originally issued by the backend after verifying the user's credentials against the database.

2.5 Context API and AuthContext

The Context API is a built in feature of React that allows data to be shared across all components without passing it manually through every level. In SkillLink, the Context API is used to manage the authentication state of the user.

The `AuthContext` file is the heart of this system. It stores three things: the current user object, the JWT token, and a loading flag. These three things are made available to every component in the application.

Listing 2.3: AuthContext state management

```
1 export function AuthProvider({ children }) {
2   const [user, setUser]      = useState(null);
3   const [token, setToken]    = useState(null);
4   const [loading, setLoading] = useState(true);
5
6   useEffect(() => {
7     const storedToken = localStorage.getItem('sl_token');
8     const storedUser  = localStorage.getItem('sl_user');
9     if (storedToken && storedUser) {
10       setToken(storedToken);
11       setUser(JSON.parse(storedUser));
12     }
13     setLoading(false);
14   });
15 }
```

```
14    }, []);
```

When the application first loads, the AuthContext checks local storage for a saved token and user object. If they exist, it means the user was already logged in from a previous session. The token is then used in every subsequent Axios request to authorize database operations on the backend.

The user object stored in the Context contains the user's ID, name, email, and role. The role field is particularly important because it determines which pages the user can see and what data the backend returns for them.

2.6 JWT Tokens

JWT stands for JSON Web Token. It is a compact and secure way of passing information between the frontend and the backend. In SkillLink, the JWT token is issued by the backend when a user successfully logs in.

The token contains three pieces of information that are encoded inside it: the user ID, the user email, and the user role. These are called claims. The backend signs the token with a secret key so that it cannot be tampered with.

When the frontend receives the token after login, it stores it in the browser's local storage. From that point on, every Axios request includes this token in the Authorization header. The backend reads the token, verifies it using the secret key, and extracts the user ID and role to decide what database query to run and what data to return.

Listing 2.4: Token storage after successful login

```
1  const handleLogin = async () => {
2    const res = await axios.post(
3      'http://localhost:5000/api/auth/login',
4      { email, password }
5    );
6
7    login(
8      {
9        user_id: res.data.user_id,
10       name:    res.data.name,
11       email:   res.data.email,
12       role:    res.data.role
13     },
14     res.data.token
15   );
```



```
16  
17   window.location.href = '/dashboard';  
18 };
```

The login function from AuthContext stores the token and user object in local storage and also updates the state so the rest of the application knows the user is now logged in.

2.7 Local Storage

Local storage is a feature of the browser that allows small amounts of data to be saved even after the browser tab is closed. SkillLink uses local storage to persist the JWT token and user information between sessions.

When a user logs in, the token is saved to local storage under the key `sl_token` and the user object is saved under `sl_user`. When the user closes the browser and comes back later, the AuthContext reads these values from local storage and restores the session automatically without requiring the user to log in again.

When the user clicks Sign Out, the logout function in AuthContext removes both items from local storage and clears the state, effectively ending the session.

Listing 2.5: Logout function clearing local storage

```
1 const logout = () => {  
2   localStorage.removeItem('sl_token');  
3   localStorage.removeItem('sl_user');  
4   setToken(null);  
5   setUser(null);  
6   window.location.href = '/landing';  
7 };
```

2.8 Summary

This chapter covered the main tools used by the frontend to communicate with the backend and ultimately with the Oracle database. React builds and displays the interface. Axios sends and receives data through HTTP requests. React Router DOM manages navigation between pages. The Context API keeps the authentication state available everywhere. JWT tokens carry the user identity with every request. And local storage keeps the session alive between browser sessions. Together these tools form the complete communication layer of the SkillLink frontend.

Chapter 3

Project Structure

3.1 Overview

Before going into the details of how each page works and how data flows between the frontend and the database, it is important to understand how the frontend project is organized. A well organized project structure makes it easier to find files, understand what each file does, and make changes without breaking other parts of the application.

The SkillLink frontend follows a standard React project structure. All the source code lives inside a folder called `src`. Inside this folder, the code is divided into four main categories: pages, components, context, and the main application files.

3.2 Folder Structure

The complete folder structure of the frontend is as follows:

Listing 3.1: Frontend Folder Structure

```
1 frontend/  
2   src/  
3     pages/  
4       admin/  
5         AdminLogin.js  
6         AdminDashboard.js  
7         AdminUsers.js  
8         AdminProjects.js  
9         AdminBids.js  
10        AdminContracts.js  
11        AdminPayments.js  
12        AdminReviews.js  
13        AdminSkills.js  
14        AdminDisputes.js  
15        AdminNotifications.js
```

```
16     Login.js
17     Register.js
18     Landing.js
19     Dashboard.js
20     Projects.js
21     Bids.js
22     Contracts.js
23     Payments.js
24     Reviews.js
25     Disputes.js
26     Notifications.js
27 components/
28     Sidebar.js
29     AdminSidebar.js
30 context/
31     AuthContext.js
32 App.js
33 App.css
34 index.css
35 index.js
```

3.3 The Pages Folder

The pages folder contains all the main screens of the application. Each file in this folder represents one complete page that the user can navigate to. There are two groups of pages inside this folder.

The first group is the regular user pages. These are the pages that clients and freelancers use. There are eleven files in this group covering everything from login and registration to projects, bids, contracts, payments, reviews, disputes, and notifications.

The second group is the admin pages. These are kept inside a separate subfolder called admin. There are eleven admin pages as well, each one showing a complete view of one table or a set of joined tables from the Oracle database. The admin pages are only accessible to users who have logged in through the admin login page.

3.4 The Components Folder

The components folder contains reusable pieces of the interface that appear on multiple pages. In SkillLink there are two components.

The first is `Sidebar.js`. This is the navigation panel that appears on the left side of every regular user page. It shows different links depending on whether the logged in user is a client or a freelancer. This role information comes from the JWT token which originally came from the database when the user logged in.

The second is `AdminSidebar.js`. This is a separate navigation panel for the admin panel. It has its own set of links pointing to all ten admin data pages. It uses an orange color theme to visually distinguish the admin panel from the regular user interface.

3.5 The Context Folder

The context folder contains only one file called `AuthContext.js`. This file is one of the most important files in the entire frontend because it manages the authentication state of the application.

`AuthContext.js` is responsible for storing the logged in user's information and JWT token, making them available to every other file in the project, handling login by saving the token to local storage, and handling logout by clearing the token from local storage and redirecting to the landing page.

Every page that needs to make an authenticated request to the backend imports the `useAuth` hook from this file to get the current token and user information.

3.6 The Main Application Files

There are three main files at the top level of the `src` folder.

`index.js` is the entry point of the entire React application. It is the first file that runs when the application starts. It renders the `App` component into the HTML page.

Listing 3.2: `index.js` entry point

```
1 const root = ReactDOM.createRoot(  
2   document.getElementById('root')  
3 );  
4 root.render(  
5   <React.StrictMode>  
6     <App />  
7   </React.StrictMode>  
8 );
```

`App.js` is the main routing file. It defines all the URL paths in the application and maps each path to the correct page component. It also defines the protected route logic that

prevents unauthenticated users from accessing dashboard pages.

index.css contains the global styles for the entire application. It defines CSS variables for colors, spacing, and typography that are used consistently across all pages. It also contains the base styles for common elements like inputs, buttons, and tables.

3.7 The App.js File in Detail

The App.js file is worth looking at in more detail because it contains the routing and access control logic that is directly tied to the database. Specifically it contains two protected route components.

The first is PrivateRoute. This component wraps all the regular user pages. When a user tries to visit a dashboard page, PrivateRoute checks whether there is a valid user in the AuthContext. If there is no user, which means no valid token from the database, the user is redirected to the landing page.

Listing 3.3: PrivateRoute component in App.js

```
1 function PrivateRoute({ children }) {  
2   const { user, loading } = useAuth();  
3   if (loading) return null;  
4   if (!user) return <Navigate to="/landing" />;  
5   return children;  
6 }
```

The second is AdminRoute. This component wraps all the admin pages. It checks not only whether a user is logged in but also whether that user has the role of admin. The role field comes from the JWT token which was set by the backend based on the admins table in the Oracle database.

Listing 3.4: AdminRoute component in App.js

```
1 function AdminRoute({ children }) {  
2   const { user, loading } = useAuth();  
3   if (loading) return null;  
4   if (!user || user.role !== "admin")  
5     return <Navigate to="/admin/login" />;  
6   return children;  
7 }
```

The UserLayout and AdminLayout components define the overall layout of the two sections of the application. Each layout wraps its pages with the appropriate sidebar and content area.

Listing 3.5: UserLayout and AdminLayout in App.js

```
1 function UserLayout() {
2   return (
3     <PrivateRoute>
4       <div style={{ display: "flex" }}>
5         <Sidebar />
6         <div style={{ marginLeft: "240px", padding: "40px" }}>
7           <Routes>
8             <Route index element={<Dashboard />} />
9             <Route path="projects" element={<Projects />} />
10            <Route path="bids" element={<Bids />} />
11            <Route path="contracts" element={<Contracts />} />
12            <Route path="payments" element={<Payments />} />
13            <Route path="reviews" element={<Reviews />} />
14            <Route path="disputes" element={<Disputes />} />
15            <Route path="notifications"
16              element={<Notifications />} />
17          </Routes>
18        </div>
19      </div>
20    </PrivateRoute>
21  );
22 }
```

3.8 How the Structure Relates to the Database

Each page in the pages folder corresponds to one or more tables in the Oracle database. The table below shows this relationship clearly.

3.9 Summary

The frontend project is organized into four main areas. The pages folder contains all the screens for both regular users and the admin. The components folder contains the sidebar navigation panels. The context folder contains the authentication state manager. And the main application files handle routing and access control. Every page in the frontend is directly connected to one or more tables in the Oracle database, and the project structure makes this connection clear and easy to follow.

Frontend Page	Database Tables Involved
Login.js	users, client_profile, freelancer_profile
Register.js	users, client_profile, freelancer_profile
Dashboard.js	projects, bids, contracts, notifications
Projects.js	projects, categories, project_category
Bids.js	bids, projects, freelancer_profile
Contracts.js	contracts, bids, projects, users
Payments.js	payments, contracts, bids, projects
Reviews.js	reviews, contracts, bids, projects, users
Disputes.js	disputes, contracts, bids, projects
Notifications.js	notifications, users
AdminDashboard.js	All 15 tables (aggregate counts)
AdminUsers.js	users, client_profile, freelancer_profile
AdminProjects.js	projects, client_profile, categories
AdminBids.js	bids, projects, freelancer_profile
AdminContracts.js	contracts, bids, projects, users
AdminPayments.js	payments, contracts, bids, projects, users
AdminReviews.js	reviews, users, contracts, projects
AdminSkills.js	user_skills, skills, freelancer_profile
AdminDisputes.js	disputes, users, contracts, projects
AdminNotifications.js	notifications, users

Table 3.1: Frontend Pages and their Corresponding Database Tables

Chapter 4

Authentication Flow

4.1 Overview

Authentication is the process of verifying who a user is before allowing them to access the system. In SkillLink, authentication is handled through a combination of the frontend forms, the backend API, and the Oracle database. This chapter explains exactly how a user registers and logs in, how the system checks their identity, and how that identity is carried through every subsequent request to the database.

4.2 The Registration Process

When a new user wants to join SkillLink, they go to the registration page. The registration page is built in two steps. In the first step, the user chooses their role. They can either be a client who posts projects and hires freelancers, or a freelancer who browses projects and places bids. This role selection is important because it determines which table in the database will receive the user's profile data.

In the second step, the user fills in their personal details. The fields shown in this step depend on the role chosen in the first step. If the user chose client, they see fields for their full name, email, phone, company name, location, and password. If they chose freelancer, they see fields for their full name, email, phone, bio, hourly rate, experience level, and password.

When the user clicks the Create Account button, the frontend collects all the filled in values and sends them to the backend in a single POST request.

Listing 4.1: Registration POST request in Register.js

```
1 const handleRegister = async () => {  
2   if (!name || !email || !password) {  
3     setError('Name, email and password are required');  
4     return;  
5   }  
}
```



```
6   setError('');
7   setLoading(true);
8   try {
9     const res = await axios.post(
10      'http://localhost:5000/api/auth/register',
11      {
12        name,
13        email,
14        password,
15        phone,
16        role,
17        company_name:    companyName,
18        location,
19        bio,
20        hourly_rate:    hourlyRate,
21        experience_level: experienceLevel,
22      }
23    );
24    login(
25      {
26        user_id: res.data.user_id,
27        name,
28        email,
29        role
30      },
31      res.data.token
32    );
33    window.location.href = '/dashboard';
34  } catch (err) {
35    setError(err.response?.data?.error || 'Registration failed');
36  } finally {
37    setLoading(false);
38  }
39 };
```

The backend receives this POST request and does the following things in order. First it checks whether the email already exists in the users table. If it does, it sends back a 409 conflict error and the frontend shows the message Email already registered. If the email is new, the backend inserts a new row into the users table using the next value from the seq_users sequence. It then inserts a row into either the client_profile table or the freelancer_profile table depending on the role that was sent. Finally it creates a JWT

token and sends it back to the frontend.

When the frontend receives the successful response, it calls the login function from `AuthContext` which saves the token and user object to local storage. The user is then redirected to the dashboard automatically.

4.3 The Login Process

The login page is simpler than the registration page. It only has two fields, email and password. When the user clicks Sign In, the frontend sends a POST request to the backend login route.

Listing 4.2: Login POST request in `Login.js`

```
1 const handleLogin = async () => {
2   if (!email || !password) {
3     setError('Please fill in both fields');
4     return;
5   }
6   setError('');
7   setLoading(true);
8   try {
9     const res = await axios.post(
10      'http://localhost:5000/api/auth/login',
11      { email, password }
12    );
13    login(
14      {
15        user_id: res.data.user_id,
16        name:    res.data.name,
17        email:   res.data.email,
18        role:    res.data.role
19      },
20      res.data.token
21    );
22    window.location.href = '/dashboard';
23  } catch (err) {
24    setError(err.response?.data?.error || 'Login failed');
25  } finally {
26    setLoading(false);
27  }
28 };
```

The backend receives this request and runs a SELECT query on the users table joined with the client_profile and freelancer_profile tables. This join is necessary because the role of the user is not stored directly in the users table. Instead it is determined by which profile table has a matching record for that user ID. If the user has a record in client_profile they are a client. If they have a record in freelancer_profile they are a freelancer.

If the email is not found in the database, the backend returns a 401 error and the frontend shows Invalid email or password. If the email is found but the password does not match, the same error is shown. This is intentional so that potential attackers cannot tell whether the email exists or not.

If both the email and password are correct, the backend creates a JWT token containing the user ID, email, and role, and sends it back along with the user's name and other details. The frontend saves everything to local storage and redirects the user to the dashboard.

4.4 The Admin Login Process

The admin login works similarly to the regular user login but uses a completely separate route and a separate database table. The admin login page is accessible at the path /admin/login and it sends its POST request to a different backend endpoint.

Listing 4.3: Admin login POST request in AdminLogin.js

```
1 const handleLogin = async () => {
2   if (!email || !password) {
3     setError('Please fill in both fields');
4     return;
5   }
6   setError('');
7   setLoading(true);
8   try {
9     const res = await axios.post(
10      'http://localhost:5000/api/admin/auth/login',
11      { email, password }
12    );
13    login(
14      {
15        user_id: res.data.admin_id,
16        name:    res.data.name,
17        email:   res.data.email,
18        role:    'admin'
19      },
```

```
20     res.data.token
21   );
22   window.location.href = '/admin';
23 } catch (err) {
24   setError(err.response?.data?.error || 'Login failed');
25 } finally {
26   setLoading(false);
27 }
28 };
```

The backend checks the admins table instead of the users table. The admins table is a separate table created specifically for this purpose. It stores the admin ID, name, email, password, and creation date. If the credentials match a record in the admins table, the backend issues a JWT token with the role set to admin. The frontend saves this token and redirects to the admin dashboard.

4.5 The AuthContext Login and Logout Functions

The AuthContext file contains two functions that the login and logout pages use directly. The login function saves the token and user data, and the logout function clears them.

Listing 4.4: Login and logout functions in AuthContext.js

```
1  const login = (userData, tokenData) => {
2    localStorage.setItem('sl_token', tokenData);
3    localStorage.setItem(
4      'sl_user',
5      JSON.stringify(userData)
6    );
7    setToken(tokenData);
8    setUser(userData);
9  };
10
11 const logout = () => {
12   localStorage.removeItem('sl_token');
13   localStorage.removeItem('sl_user');
14   setToken(null);
15   setUser(null);
16   window.location.href = '/landing';
17 };
```

The login function takes two arguments. The first is the user data object which contains

the user ID, name, email, and role. The second is the JWT token string. Both are saved to local storage so they survive a browser refresh or a closed tab. The state variables are also updated so the rest of the application knows immediately that someone is now logged in.

The logout function does the opposite. It removes both items from local storage and sets the state variables back to null. The user is then redirected to the landing page. At this point the application no longer has a token so any attempt to access a protected page will redirect back to the landing page.

4.6 Session Persistence

One important feature of the authentication system is that it remembers the user between sessions. When the application first loads, the AuthContext runs a `useEffect` hook that checks local storage for a saved token and user object.

Listing 4.5: Session restoration in AuthContext.js

```
1 useEffect(() => {  
2   const storedToken = localStorage.getItem('sl_token');  
3   const storedUser = localStorage.getItem('sl_user');  
4   if (storedToken && storedUser) {  
5     setToken(storedToken);  
6     setUser(JSON.parse(storedUser));  
7   }  
8   setLoading(false);  
9 }, []);
```

If both values are found in local storage, they are loaded into the state. This means that if a user logged in yesterday and comes back today, they will go straight to the dashboard without having to log in again. The loading flag is set to true while this check is happening and then set to false when it is done. During the loading phase, the `PrivateRoute` and `AdminRoute` components return null, which means they show nothing while waiting for the authentication check to complete. This prevents a brief flash where the user sees the login page before being redirected to the dashboard.

4.7 How Authentication Ties to the Database

The entire authentication system is rooted in the database. The `users` table stores the credentials of all clients and freelancers. The `admins` table stores the credentials of all admins. The `client_profile` and `freelancer_profile` tables determine the role of each user.

When a user logs in, the backend runs this query against the database:

Listing 4.6: Backend login query joining three tables

```
1 SELECT u.user_id, u.name, u.email, u.password,
2     CASE
3         WHEN c.client_id IS NOT NULL THEN 'client'
4         WHEN f.freelancer_id IS NOT NULL THEN 'freelancer'
5     END as role
6 FROM users u
7 LEFT JOIN client_profile c ON u.user_id = c.client_id
8 LEFT JOIN freelancer_profile f
9     ON u.user_id = f.freelancer_id
10 WHERE u.email = :email
```

This query joins three tables to find the user and determine their role in a single database call. The result comes back to the backend, which creates the JWT token from it and sends it to the frontend. The frontend stores this token and uses it for every future request. In this way, the single login query shapes everything that happens in the frontend session that follows.

4.8 Role Based Redirection

After a successful login, the frontend redirects the user based on their role. Regular users go to /dashboard and admins go to /admin. Inside the dashboard, the Sidebar component reads the role from the AuthContext and shows different navigation links depending on whether the user is a client or a freelancer.

Listing 4.7: Role based sidebar links in Sidebar.js

```
1 const clientLinks = [
2     { to: '/dashboard',          label: 'Dashboard'      },
3     { to: '/dashboard/projects', label: 'My Projects'   },
4     { to: '/dashboard/bids',     label: 'Bids Received' },
5     { to: '/dashboard/contracts', label: 'Contracts'     },
6     { to: '/dashboard/payments', label: 'Payments'      },
7     { to: '/dashboard/reviews',  label: 'Reviews'       },
8     { to: '/dashboard/disputes', label: 'Disputes'      },
9     { to: '/dashboard/notifications', label: 'Notifications' },
10 ];
11
12 const freelancerLinks = [
13     { to: '/dashboard',          label: 'Dashboard'      },
```

```
14 { to: '/dashboard/projects',      label: 'Browse Projects' },
15 { to: '/dashboard/bids',         label: 'My Bids' },
16 { to: '/dashboard/contracts',    label: 'Contracts' },
17 { to: '/dashboard/payments',     label: 'Earnings' },
18 { to: '/dashboard/reviews',      label: 'Reviews' },
19 { to: '/dashboard/disputes',     label: 'Disputes' },
20 { to: '/dashboard/notifications', label: 'Notifications' },
21 ];
22
23 const links = role === 'client' ? clientLinks : freelancerLinks;
```

The role value that drives this decision originally came from the database at the time of login. It was encoded in the JWT token and stored in local storage. Every time the sidebar renders, it reads this value from the AuthContext and uses it to decide which set of links to show.

4.9 Summary

The authentication flow in SkillLink connects the frontend directly to the database at the very first moment a user interacts with the system. Registration inserts new rows into the users and profile tables. Login reads from those tables to verify credentials and determine the user role. The JWT token carries that role information through every page visit and every data request that follows. The admin login uses a separate table and a separate set of pages to keep the admin access completely isolated from regular user access. The entire system is designed so that the database is always the source of truth for who a user is and what they are allowed to do.

Chapter 5

Data Fetching: SELECT Operations

5.1 Overview

Data fetching refers to the process of reading data from the database and displaying it on the screen. In SkillLink, every page that shows information to the user is performing a data fetch behind the scenes. The frontend sends a GET request to the backend, the backend runs a SELECT query against the Oracle database, and the result comes back to the frontend where it is displayed on the screen.

This chapter goes through each page of the frontend and explains exactly what data it fetches, which database tables are involved, and how the fetched data is displayed to the user.

5.2 How Data Fetching Works in React

Before going into each page, it is worth understanding the general pattern used for data fetching across the entire application. Every page that needs data from the database follows the same four steps.

The first step is to define state variables to hold the data. These are empty at first and get filled when the data arrives.

The second step is to use a `useEffect` hook that runs when the page first loads. Inside this hook, the Axios GET request is made to the backend.

The third step is to wait for the response. When it arrives, the data is stored in the state variables using the setter functions.

The fourth step is to display the data. The JSX in the component reads from the state variables and renders the data on the screen. If the data has not arrived yet, a loading message is shown instead.

The code below shows this general pattern that is repeated across all pages:

Listing 5.1: General data fetching pattern used across all pages

```
1 const [data, setData] = useState([]);
2 const [loading, setLoading] = useState(true);
3 const headers = { Authorization: 'Bearer ${token}' };
4
5 useEffect(() => { fetchData(); }, []);
6
7 const fetchData = async () => {
8   try {
9     setLoading(true);
10    const res = await axios.get(
11      `${API}/endpoint`,
12      { headers }
13    );
14    setData(res.data);
15  } catch (err) {
16    console.error(err);
17  } finally {
18    setLoading(false);
19  }
20 };
```

5.3 Dashboard Page

The dashboard is the first page a user sees after logging in. It shows a summary of the user's activity on the platform. Because the dashboard shows data from multiple tables at once, it makes several requests in parallel using `Promise.all`, which sends all requests at the same time and waits for all of them to finish before updating the screen.

The dashboard behaves differently depending on whether the logged in user is a client or a freelancer.

5.3.1 Client Dashboard

When a client logs in and opens the dashboard, the frontend makes two parallel GET requests. The first fetches all projects posted by that client. The second fetches the count of unread notifications for that client.

Listing 5.2: Client dashboard data fetching in `Dashboard.js`

```
1 const [projRes, notifRes] = await Promise.all([
2   axios.get(
```

```
3     `${API}/projects/client/${user.user_id}`,
4     { headers }
5   ),
6   axios.get(
7     `${API}/notifications/unread-count`,
8     { headers }
9   ),
10  ]);
11  const projs = projRes.data;
12  setProjects(projs.slice(0, 5));
13  const totalBids = projs.reduce(
14    (sum, p) => sum + (p[7] || 0), 0
15  );
16  setStats({
17    projects:      projs.length,
18    bids:          totalBids,
19    contracts:     projs.filter(p =>
20      p[5] === 'in_progress' ||
21      p[5] === 'completed'
22    ).length,
23    notifications: notifRes.data.count,
24  });
```

The backend responds to the first request by running a SELECT query on the projects table filtered by the client ID. It also counts the bids on each project using a subquery on the bids table. The frontend uses this data to calculate the four stat cards shown on the dashboard: total projects, total bids received, active contracts, and unread notifications.

The database tables involved in the client dashboard are projects, bids, and notifications.

5.3.2 Freelancer Dashboard

When a freelancer opens the dashboard, the frontend makes three parallel GET requests. The first fetches all open projects from the platform. The second fetches all bids submitted by that freelancer. The third fetches the unread notification count.

Listing 5.3: Freelancer dashboard data fetching in Dashboard.js

```
1  const [projRes, bidRes, notifRes] = await Promise.all([
2    axios.get(`${API}/projects`),
3    axios.get(
4      `${API}/bids/freelancer/${user.user_id}`,
5      { headers }
```

```
6   ),
7   axios.get(
8     `${API}/notifications/unread-count`,
9     { headers }
10  ),
11 ];
12 setProjects(
13   projRes.data.filter(p => p[5] === 'open').slice(0, 5)
14 );
15 setBids(bidRes.data.slice(0, 5));
16 setStats({
17   projects: projRes.data.filter(
18     p => p[5] === 'open'
19   ).length,
20   bids:      bidRes.data.length,
21   contracts: bidRes.data.filter(
22     b => b[3] === 'accepted'
23   ).length,
24   notifications: notifRes.data.count,
25 });
```

The database tables involved in the freelancer dashboard are projects, bids, users, and notifications.

5.4 Projects Page

The Projects page shows a different view depending on the role of the logged in user.

5.4.1 Client Projects View

When a client opens the Projects page, the frontend fetches only the projects that belong to that specific client. The URL includes the client's user ID so the backend knows which projects to return.

Listing 5.4: Client projects fetch in Projects.js

```
1 const url = role === 'client'
2   ? `${API}/projects/client/${user.user_id}`
3   : `${API}/projects`;
4 const res = await axios.get(
5   url,
6   role === 'client' ? { headers } : {}
```

```
7 );  
8 setProjects(res.data);
```

The backend runs a SELECT query on the projects table filtered by the client ID. It also joins with the project_category and categories tables to get the category name, and uses a subquery to count the total bids and pending bids on each project. The result gives the client a complete picture of each of their projects including how many bids they have received.

5.4.2 Freelancer Projects View

When a freelancer opens the Projects page, the frontend fetches all projects from the platform regardless of who posted them. The freelancer can then browse through them and decide which ones to bid on.

The backend responds by running a SELECT query on the projects table joined with the users table to get the client name, the project_category table, and the categories table to get the category name. It also counts the total bids on each project using a subquery on the bids table.

The database tables involved in the Projects page are projects, users, project_category, categories, and bids.

5.5 Bids Page

The Bids page also shows different data depending on the role of the user.

5.5.1 Client Bids View

When a client opens the Bids page, the frontend first fetches all the projects belonging to that client. It then automatically loads the bids for the first project in the list.

Listing 5.5: Client bids data fetching in Bids.js

```
1 const projRes = await axios.get(  
2   `${API}/projects/client/${user.user_id}`,  
3   { headers }  
4 );  
5 setProjects(projRes.data);  
6 if (projRes.data.length > 0) {  
7   setSelected(projRes.data[0][0]);  
8   const bidRes = await axios.get(  
9     `${API}/bids/project/${projRes.data[0][0]}`,
```

```
10     { headers }
11   );
12   setBids (bidRes.data);
13 }
```

When the client clicks on a different project in the list, the frontend makes another GET request to fetch the bids for that project.

Listing 5.6: Fetching bids for a selected project

```
1 const fetchBidsForProject = async (projectId) => {
2   setSelected(projectId);
3   const res = await axios.get(
4     `${API}/bids/project/${projectId}`,
5     { headers }
6   );
7   setBids(res.data);
8 };
```

The backend responds by running a SELECT query on the bids table joined with the users table to get the freelancer name and email, and the freelancer_profile table to get the rating, experience level, and bio. This gives the client everything they need to evaluate each bid.

5.5.2 Freelancer Bids View

When a freelancer opens the Bids page, the frontend fetches two things in parallel. The first is all open projects so the freelancer can select one to bid on. The second is all bids previously submitted by that freelancer.

Listing 5.7: Freelancer bids data fetching in Bids.js

```
1 const [projRes, bidRes] = await Promise.all([
2   axios.get(`${API}/projects`),
3   axios.get(
4     `${API}/bids/freelancer/${user.user_id}`,
5     { headers }
6   ),
7 ]);
8 setProjects(projRes.data.filter(p => p[5] === 'open'));
9 setBids(bidRes.data);
```

The backend responds to the freelancer bids request by running a SELECT query on the bids table joined with the projects table to get the project title and budget, and the users

table to get the client name. This gives the freelancer a full history of all their bids along with the current status of each one.

The database tables involved in the Bids page are bids, projects, users, and freelancer_profile.

5.6 Contracts Page

The Contracts page fetches the contracts that belong to the logged in user. The same endpoint is used for both clients and freelancers, but the backend runs a different query depending on the role stored in the JWT token.

Listing 5.8: Contracts data fetching in Contracts.js

```
1 const res = await axios.get(  
2   `${API}/contracts/my`,  
3   { headers }  
4 );  
5 setContracts(res.data);
```

For a client, the backend joins the contracts table with the bids table, the projects table, and the users table to get the freelancer's name and email. For a freelancer, the same tables are joined but from the other direction to get the client's name and email.

Each contract row contains the contract ID, amount, start date, end date, status, project title, and the name of the other party. The frontend uses all of these fields to display a complete contract card for each contract.

The database tables involved in the Contracts page are contracts, bids, projects, users, client_profile, and freelancer_profile.

5.7 Payments Page

The Payments page fetches two sets of data in parallel. The first is the payment history for the logged in user. The second is the list of active contracts, which is needed to populate the payment form for clients.

Listing 5.9: Payments data fetching in Payments.js

```
1 const [payRes, contRes] = await Promise.all([  
2   axios.get(`${API}/payments/my`, { headers }),  
3   axios.get(`${API}/contracts/my`, { headers }),  
4 ]);  
5 setPayments(payRes.data);  
6 setContracts(contRes.data.filter(c => c[4] === 'active'));
```

The backend responds to the payments request by joining the payments table with the contracts table, the bids table, the projects table, and the users table. For a client this gives the freelancer name and project title for each payment. For a freelancer this gives the client name and project title.

The frontend uses the payment data to calculate the total amount spent or earned and displays a summary card at the top of the page followed by a detailed table of all transactions.

The database tables involved in the Payments page are payments, contracts, bids, projects, users, client_profile, and freelancer_profile.

5.8 Reviews Page

The Reviews page fetches three sets of data in parallel. The first is all reviews received by the logged in user. The second is all reviews given by the logged in user. The third is all completed contracts, which is needed to populate the review form.

Listing 5.10: Reviews data fetching in Reviews.js

```
1 const [recRes, givRes, contRes] = await Promise.all([
2   axios.get(`${API}/reviews/my`, { headers }),
3   axios.get(`${API}/reviews/given`, { headers }),
4   axios.get(`${API}/contracts/my`, { headers }),
5 ]);
6 setReceived(recRes.data);
7 setGiven(givRes.data);
8 setContracts(contRes.data.filter(c => c[4] === 'completed'));
```

The backend responds to the reviews requests by joining the reviews table with the users table to get the reviewer and reviewee names, and the contracts, bids, and projects tables to get the project title. The frontend displays the reviews in two tabs, one for received reviews and one for given reviews.

The database tables involved in the Reviews page are reviews, users, contracts, bids, and projects.

5.9 Disputes Page

The Disputes page fetches two sets of data in parallel. The first is all disputes related to the logged in user. The second is all active contracts, which is needed to populate the dispute form.

Listing 5.11: Disputes data fetching in Disputes.js

```
1 const [disRes, contRes] = await Promise.all([
2   axios.get(`${API}/disputes/my`, { headers }),
3   axios.get(`${API}/contracts/my`, { headers }),
4 ]);
5 setDisputes(disRes.data);
6 setContracts(contRes.data.filter(c => c[4] === 'active'));
```

The backend responds to the disputes request by joining the disputes table with the users table to get the name of the person who raised the dispute, the contracts table to get the contract amount, the bids table, and the projects table to get the project title.

The database tables involved in the Disputes page are disputes, users, contracts, bids, and projects.

5.10 Notifications Page

The Notifications page fetches all notifications belonging to the logged in user. This is one of the simpler fetch operations in the application because it only involves one table directly.

Listing 5.12: Notifications data fetching in Notifications.js

```
1 const res = await axios.get(
2   `${API}/notifications/my`,
3   { headers }
4 );
5 setNotifications(res.data);
```

The backend responds by running a SELECT query on the notifications table filtered by the user ID from the JWT token. Each notification row contains the notification ID, message, date sent, read status, and type. The frontend uses the read status field to show unread notifications with a teal left border and a blue dot indicator. Read notifications are shown at reduced opacity.

The database tables involved in the Notifications page are notifications and users.

5.11 Summary of Data Fetching Operations

The table below summarizes all the data fetching operations performed by the frontend and the database tables involved in each one.

Page	HTTP Method	Database Tables
Dashboard (Client)	GET	projects, bids, notifications
Dashboard (Freelancer)	GET	projects, bids, users, notifications
Projects (Client)	GET	projects, project_category, categories, bids
Projects (Freelancer)	GET	projects, users, project_category, categories, bids
Bids (Client)	GET	bids, users, freelancer_profile
Bids (Freelancer)	GET	bids, projects, users
Contracts	GET	contracts, bids, projects, users
Payments	GET	payments, contracts, bids, projects, users
Reviews	GET	reviews, users, contracts, bids, projects
Disputes	GET	disputes, users, contracts, bids, projects
Notifications	GET	notifications

Table 5.1: Summary of Frontend Data Fetching Operations

5.12 Summary

Every page in the SkillLink frontend is connected to the Oracle database through a chain of GET requests and SELECT queries. The frontend never reads from the database directly. Instead it sends a request to the backend, the backend runs the appropriate query, and the result comes back to the frontend where it is stored in state variables and displayed on the screen. The use of Promise.all on pages that need multiple sets of data at once makes the fetching efficient by running all the queries in parallel rather than one after another.

Chapter 6

Data Sending: INSERT and UPDATE Operations

6.1 Overview

While the previous chapter covered how the frontend reads data from the database through GET requests and SELECT queries, this chapter covers the opposite direction. Data sending refers to the process of the user filling in a form on the frontend and that data being sent to the backend which then runs an INSERT or UPDATE query against the Oracle database.

In SkillLink there are six main actions that send data to the database. These are registering a new account, posting a project, placing a bid, accepting a bid, recording a payment, submitting a review, and raising a dispute. Each of these actions is triggered by a button click on the frontend, sends a POST or PUT request to the backend, and results in a change to one or more tables in the Oracle database.

6.2 How Data Sending Works in React

The general pattern for sending data follows the same structure across all pages. The user fills in form fields whose values are stored in state variables. When the submit button is clicked, a handler function runs that first validates the input, then sends the data to the backend using Axios, and finally shows a success or error message based on the response.

Listing 6.1: General pattern for data sending

```
1 const handleSubmit = async () => {  
2   if (!requiredField) {  
3     setError('Field is required');  
4     return;  
5   }  
6   setError('');
```

```
7   setSubmitting(true);
8   try {
9     await axios.post(`${API}/endpoint`, formData, { headers });
10    setSuccess('Action completed successfully!');
11    fetchData();
12  } catch (err) {
13    setError(err.response?.data?.error || 'Action failed');
14  } finally {
15    setSubmitting(false);
16  }
17 };
```

6.3 Registration: INSERT into Three Tables

Registration is the most complex data sending operation in the frontend because it results in data being written to three tables in a single backend transaction. When a new user submits the registration form, the frontend sends a POST request with all the form values including the role, name, email, phone, password, and role specific fields.

Listing 6.2: Registration data sent in Register.js

```
1  const res = await axios.post(
2    'http://localhost:5000/api/auth/register',
3    {
4      name,
5      email,
6      password,
7      phone,
8      role,
9      company_name:    companyName,
10     location,
11     bio,
12     hourly_rate:     hourlyRate,
13     experience_level: experienceLevel,
14   }
15 );
```

The backend receives this data and runs three INSERT statements inside a single transaction. The first inserts a new row into the users table using the seq_users sequence to generate the primary key. The second inserts into either client_profile or freelancer_profile depending on the role value. Both inserts are wrapped in a transaction so that if either

one fails, neither one is committed to the database.

The tables affected by registration are users, client_profile, and freelancer_profile.

6.4 Posting a Project: INSERT into Projects

When a client fills in the Post a Project form on the Projects page and clicks the Post Project button, the frontend sends a POST request to the backend projects route.

Listing 6.3: Post project handler in Projects.js

```
1 const handlePost = async () => {
2   if (!title || !budget || !deadline) {
3     setError('Title, budget and deadline are required');
4     return;
5   }
6   setError('');
7   setSubmitting(true);
8   try {
9     await axios.post(
10      `${API}/projects`,
11      { title, description, budget, deadline },
12      { headers }
13    );
14    setSuccess('Project posted successfully!');
15    setShowForm(false);
16    setTitle('');
17    setDescription('');
18    setBudget('');
19    setDeadline('');
20    fetchProjects();
21    setTimeout(() => setSuccess(''), 3000);
22  } catch (err) {
23    setError(err.response?.data?.error || 'Failed to post project');
24  } finally {
25    setSubmitting(false);
26  }
27 };
```

The client ID is not sent in the form data because the backend extracts it from the JWT token. This is a security measure. If the client ID were sent from the frontend, a malicious user could change it to post projects on behalf of another user. By reading the client ID

from the verified token instead, the backend ensures that projects can only be posted by the actual logged in user.

The backend runs an INSERT into the projects table using the seq_projects sequence for the primary key. The status is automatically set to open since all new projects start in the open state.

The table affected by posting a project is projects.

6.5 Placing a Bid: INSERT into Bids

When a freelancer selects a project and enters a bid amount and delivery time, the frontend sends a POST request to the bids route.

Listing 6.4: Submit bid handler in Bids.js

```
1 const handleSubmitBid = async () => {
2   if (!bidProject || !amount || !deliveryTime) {
3     setError('All fields are required');
4     return;
5   }
6   setError('');
7   setSubmitting(true);
8   try {
9     await axios.post(
10      `${API}/bids`,
11      {
12        project_id:    bidProject,
13        amount,
14        delivery_time: deliveryTime,
15      },
16      { headers }
17    );
18    setSuccess('Bid submitted successfully!');
19    setShowForm(false);
20    setAmount('');
21    setDeliveryTime('');
22    setBidProject('');
23    fetchData();
24    setTimeout(() => setSuccess(''), 3000);
25  } catch (err) {
26    setError(err.response?.data?.error || 'Failed to submit bid')
    ;
  }
```

```
27   } finally {  
28       setSubmitting(false);  
29   }  
30 };
```

The freelancer ID is again read from the JWT token on the backend rather than being sent from the frontend. The backend also runs two validation checks before inserting the bid. The first check confirms that the project status is open, meaning it is still accepting bids. The second check confirms that the same freelancer has not already placed a bid on that project. If either check fails, the backend sends an error response and the frontend displays it to the user. If both checks pass, the backend inserts a new row into the bids table with a status of pending.

The table affected by placing a bid is bids.

6.6 Accepting a Bid: UPDATE Four Tables

Accepting a bid is the most complex UPDATE operation in the entire application because it triggers changes in four different tables at once. When a client clicks the Accept Bid button next to a bid on the Bids page, the frontend sends a PUT request to the backend.

Listing 6.5: Accept bid handler in Bids.js

```
1  const handleAccept = async (bidId) => {  
2      if (!window.confirm(  
3          'Accept this bid? A contract will be created automatically.'  
4      )) return;  
5      try {  
6          await axios.put(  
7              `${API}/bids/${bidId}/accept`,  
8              {},  
9              { headers }  
10         );  
11         setSuccess('Bid accepted! Contract created successfully.');
```

```
12         fetchBidsForProject(selected);  
13         setTimeout(() => setSuccess(''), 4000);  
14     } catch (err) {  
15         setError(err.response?.data?.error || 'Failed to accept bid')  
16         ;  
17         setTimeout(() => setError(''), 4000);  
18     }  
19 };
```

The backend receives this PUT request and runs four database operations inside a single transaction. The first UPDATE sets the status of the accepted bid to accepted. The second UPDATE sets the status of all other bids on the same project to rejected, so that only one bid can be accepted per project. The third UPDATE sets the project status from open to in_progress. The fourth operation is an INSERT into the contracts table that creates a new contract automatically, calculating the end date by adding the delivery time in days to the current date.

Because all four operations are wrapped in a transaction with a single commit at the end, either all of them succeed together or none of them are applied. This protects the database from being left in an inconsistent state if something goes wrong partway through.

The tables affected by accepting a bid are bids, projects, and contracts.

6.7 Recording a Payment: INSERT and UPDATE

When a client records a payment for a completed project, the frontend sends a POST request to the payments route with the contract ID, amount, and payment method.

Listing 6.6: Record payment handler in Payments.js

```
1 const handlePay = async () => {
2   if (!contractId || !amount || !method) {
3     setError('All fields are required');
4     return;
5   }
6   setError('');
7   setSubmitting(true);
8   try {
9     await axios.post(
10      `${API}/payments`,
11      {
12        contract_id: contractId,
13        amount,
14        method,
15      },
16      { headers }
17    );
18    setSuccess(
19      'Payment recorded and contract marked as completed!'
20    );
21    setShowForm(false);
```

```
22     setContractId('');
23     setAmount('');
24     setMethod('');
25     fetchData();
26     setTimeout(() => setSuccess(''), 4000);
27   } catch (err) {
28     setError(err.response?.data?.error || 'Failed to record
      payment');
29   } finally {
30     setSubmitting(false);
31   }
32 };
```

The backend receives this request and first checks that the contract exists and is still active. If the contract has already been completed, it sends back an error. If the contract is active, the backend inserts a new row into the payments table with a status of paid, and then updates the contract status from active to completed. It also runs a further UPDATE to set the related project status to completed.

Like the bid acceptance, these operations are wrapped in a single transaction so they either all succeed or all fail together.

The tables affected by recording a payment are payments and contracts.

6.8 Submitting a Review: INSERT and UPDATE

After a contract is completed, both the client and the freelancer can leave a review for each other. The review form asks for a contract selection, a star rating from one to five, and an optional comment. When the form is submitted, the frontend sends a POST request to the reviews route.

Listing 6.7: Submit review handler in Reviews.js

```
1  const handleSubmit = async () => {
2    if (!contractId || !revieweeId || !rating) {
3      setError('Please fill all required fields');
4      return;
5    }
6    setError('');
7    setSubmitting(true);
8    try {
9      await axios.post(
10        `${API}/reviews`,
```



```
11     {
12         contract_id: contractId,
13         reviewee_id: revieweeId,
14         rating,
15         comment,
16     },
17     { headers }
18 );
19 setSuccess('Review submitted successfully!');
20 setShowForm(false);
21 setContractId('');
22 setRevieweeId('');
23 setRating(5);
24 setComment('');
25 fetchData();
26 setTimeout(() => setSuccess(''), 3000);
27 } catch (err) {
28     setError(err.response?.data?.error || 'Failed to submit
        review');
29 } finally {
30     setSubmitting(false);
31 }
32 };
```

The reviewer ID is taken from the JWT token on the backend. The backend first checks that the contract is completed before allowing a review. It also checks that the same reviewer has not already submitted a review for that reviewee on that contract. If both checks pass, the backend inserts a new row into the reviews table and then runs an UPDATE on the freelancer_profile table to recalculate the freelancer's average rating using AVG on all their reviews.

The tables affected by submitting a review are reviews and freelancer_profile.

6.9 Raising a Dispute: INSERT into Disputes

If either a client or a freelancer has a problem with an active contract, they can raise a dispute. The dispute form asks them to select an active contract and describe the reason for the dispute. When submitted, the frontend sends a POST request to the disputes route.

Listing 6.8: Raise dispute handler in Disputes.js

```
1 const handleRaise = async () => {
2   if (!contractId || !reason) {
3     setError('Contract and reason are required');
4     return;
5   }
6   setError('');
7   setSubmitting(true);
8   try {
9     await axios.post(
10      `${API}/disputes`,
11      { contract_id: contractId, reason },
12      { headers }
13    );
14    setSuccess('Dispute raised successfully. ');
15    setShowForm(false);
16    setContractId('');
17    setReason('');
18    fetchData();
19    setTimeout(() => setSuccess(''), 3000);
20  } catch (err) {
21    setError(err.response?.data?.error || 'Failed to raise
22      dispute');
23  } finally {
24    setSubmitting(false);
25  }
26 };
```

The user ID of the person raising the dispute is read from the JWT token on the backend. The backend checks that the contract exists and that there is no existing open dispute for that contract. If the checks pass, it inserts a new row into the disputes table with a status of open.

The table affected by raising a dispute is disputes.

6.10 Resolving a Dispute: UPDATE Disputes

A dispute can be marked as resolved by either party. This is done through a PUT request sent when the user clicks the Mark as Resolved button.

Listing 6.9: Resolve dispute handler in Disputes.js

```
1 const handleResolve = async (id) => {
```

```
2   if (!window.confirm(  
3       'Mark this dispute as resolved?'  
4   )) return;  
5   try {  
6       await axios.put(  
7           `${API}/disputes/${id}/resolve`,  
8           {},  
9           { headers }  
10      );  
11      setSuccess('Dispute marked as resolved.');
```

```
12      fetchData();  
13      setTimeout(() => setSuccess(''), 3000);  
14  } catch (err) {  
15      setError(  
16          err.response?.data?.error || 'Failed to resolve dispute'  
17      );  
18  }  
19  };
```

The backend receives this PUT request and runs a single UPDATE on the disputes table, changing the status from open to resolved.

The table affected by resolving a dispute is disputes.

6.11 Summary of Data Sending Operations

The table below summarizes all the data sending operations in the frontend along with the HTTP method used, the database operation triggered, and the tables affected.

6.12 Summary

Every form and action button in the SkillLink frontend connects directly to the Oracle database through the backend API. Simple actions like posting a project result in a single INSERT into one table. More complex actions like accepting a bid trigger a chain of four database operations across multiple tables, all wrapped in a single transaction to protect data integrity. The frontend's role in all of this is to collect the user's input, validate it before sending, and display the result clearly once the database operation is complete.

Action	Method	DB Operation	Tables Affected
Register	POST	INSERT	users, client_profile / freelancer_profile
Post a Project	POST	INSERT	projects
Place a Bid	POST	INSERT	bids
Accept a Bid	PUT	UPDATE + INSERT	bids, projects, contracts
Record a Payment	POST	INSERT + UPDATE	payments, contracts, projects
Submit a Review	POST	INSERT + UPDATE	reviews, freelancer_profile
Raise a Dispute	POST	INSERT	disputes
Resolve a Dispute	PUT	UPDATE	disputes

Table 6.1: Summary of Frontend Data Sending Operations

Chapter 7

Role Based Access Control

7.1 Overview

Role based access control means that different users see different things and can do different actions depending on who they are. In SkillLink there are three roles: client, freelancer, and admin. Each role has its own set of pages, its own navigation links, and its own data views. The role of a user is determined at login time by the database and is carried through the entire session inside the JWT token.

This chapter explains how the frontend uses the role to control what each user can see and do, and how this connects back to the data stored in the Oracle database.

7.2 How the Role is Determined

The role is not something the frontend decides on its own. It comes directly from the database. When a user logs in, the backend runs a SELECT query that joins the users table with the client_profile and freelancer_profile tables. If the user has a matching record in client_profile, the role is set to client. If they have a matching record in freelancer_profile, the role is set to freelancer.

Listing 7.1: Backend query that determines user role

```
1 SELECT u.user_id, u.name, u.email, u.password,
2     CASE
3         WHEN c.client_id IS NOT NULL THEN 'client'
4         WHEN f.freelancer_id IS NOT NULL THEN 'freelancer'
5     END as role
6 FROM users u
7 LEFT JOIN client_profile c ON u.user_id = c.client_id
8 LEFT JOIN freelancer_profile f
9     ON u.user_id = f.freelancer_id
10 WHERE u.email = :email
```

This role value is then encoded into the JWT token and sent to the frontend. The frontend stores it in local storage as part of the user object. From that point on, every component in the application can read the role from the AuthContext and use it to decide what to show.

Listing 7.2: Reading the role from AuthContext

```
1 const { user, token } = useAuth();
2 const role = user?.role;
```

7.3 Route Level Access Control

The first level of access control happens at the route level in App.js. There are two protected route components that act as gates to different parts of the application.

The PrivateRoute component protects all the regular user pages under the /dashboard path. It checks whether there is a logged in user in the AuthContext. If there is no user, the visitor is sent to the landing page. If there is a user, they are allowed through regardless of whether they are a client or a freelancer, because both roles share the same dashboard layout.

Listing 7.3: PrivateRoute protecting user pages in App.js

```
1 function PrivateRoute({ children }) {
2   const { user, loading } = useAuth();
3   if (loading) return null;
4   if (!user) return <Navigate to="/landing" />;
5   return children;
6 }
```

The AdminRoute component protects all the admin pages under the /admin path. It checks two things. First it checks that there is a logged in user. Second it checks that the user's role is specifically admin. A client or freelancer who tries to navigate to /admin will be redirected to the admin login page even if they are logged in as a regular user.

Listing 7.4: AdminRoute protecting admin pages in App.js

```
1 function AdminRoute({ children }) {
2   const { user, loading } = useAuth();
3   if (loading) return null;
4   if (!user || user.role !== 'admin')
5     return <Navigate to="/admin/login" />;
6   return children;
7 }
```

The admin role is set differently from the client and freelancer roles. When an admin logs in through the admin login page, the backend checks the admins table instead of the users table and sets the role to admin in the JWT token. This is a completely separate authentication path that keeps admin access isolated from regular user access.

7.4 Navigation Level Access Control

The second level of access control happens in the Sidebar component. The sidebar shows different navigation links depending on the role of the logged in user. Clients and freelancers share the same sidebar component but see different labels for some of the same pages.

Listing 7.5: Role based navigation links in Sidebar.js

```
1  const clientLinks = [  
2    { to: '/dashboard',          label: 'Dashboard'      },  
3    { to: '/dashboard/projects', label: 'My Projects'   },  
4    { to: '/dashboard/bids',     label: 'Bids Received' },  
5    { to: '/dashboard/contracts', label: 'Contracts'    },  
6    { to: '/dashboard/payments', label: 'Payments'     },  
7    { to: '/dashboard/reviews',  label: 'Reviews'     },  
8    { to: '/dashboard/disputes', label: 'Disputes'     },  
9    { to: '/dashboard/notifications', label: 'Notifications' },  
10 ];  
11  
12 const freelancerLinks = [  
13   { to: '/dashboard',          label: 'Dashboard'      },  
14   { to: '/dashboard/projects', label: 'Browse Projects' },  
15   { to: '/dashboard/bids',     label: 'My Bids'        },  
16   { to: '/dashboard/contracts', label: 'Contracts'    },  
17   { to: '/dashboard/payments', label: 'Earnings'     },  
18   { to: '/dashboard/reviews',  label: 'Reviews'     },  
19   { to: '/dashboard/disputes', label: 'Disputes'     },  
20   { to: '/dashboard/notifications', label: 'Notifications' },  
21 ];  
22  
23 const links = role === 'client' ? clientLinks : freelancerLinks;
```

For a client, the Projects link says My Projects because clients only see their own posted projects. For a freelancer, the same link says Browse Projects because freelancers see all open projects on the platform. Similarly the Payments link says Payments for a client

and Earnings for a freelancer because both go to the same page but the data shown is filtered differently based on role.

The admin has a completely separate sidebar component called AdminSidebar that shows ten links pointing to the ten admin data pages. This sidebar uses an orange color theme instead of the teal theme used in the user sidebar to visually signal that the user is in the admin area.

7.5 Page Level Access Control

The third and deepest level of access control happens inside individual pages. Many pages in SkillLink show completely different content depending on the role of the logged in user, even though the URL is the same.

7.5.1 Dashboard Page

The Dashboard page checks the role and fetches different data and shows different statistics for clients and freelancers.

Listing 7.6: Role based data fetching in Dashboard.js

```
1  if (role === 'client') {
2    const [projRes, notifRes] = await Promise.all([
3      axios.get(
4        `${API}/projects/client/${user.user_id}`,
5        { headers }
6      ),
7      axios.get(
8        `${API}/notifications/unread-count`,
9        { headers }
10     ),
11   ]);
12 } else {
13   const [projRes, bidRes, notifRes] = await Promise.all([
14     axios.get(`${API}/projects`),
15     axios.get(
16       `${API}/bids/freelancer/${user.user_id}`,
17       { headers }
18     ),
19     axios.get(
20       `${API}/notifications/unread-count`,
21       { headers }
22     ),
```



```
23   });  
24 }
```

A client's dashboard fetches their own projects and counts their bids received. A freelancer's dashboard fetches all open projects on the platform and their own bid history. The stat cards shown are also different. A client sees My Projects, Total Bids, Active Contracts, and Unread Alerts. A freelancer sees Open Projects, My Bids, Won Contracts, and Unread Alerts.

7.5.2 Projects Page

The Projects page is one of the most clearly role dependent pages in the application. When a client opens it, they see only their own projects and a form to post a new one. When a freelancer opens it, they see all open projects on the platform and a search box to find ones they want to bid on.

Listing 7.7: Role based project fetching in Projects.js

```
1 const url = role === 'client'  
2   ? `${API}/projects/client/${user.user_id}`  
3   : `${API}/projects`;  
4 const res = await axios.get(  
5   url,  
6   role === 'client' ? { headers } : {}  
7 );
```

The Post Project form is also only shown to clients. The frontend checks the role before rendering it.

Listing 7.8: Role based form visibility in Projects.js

```
1 {role === 'client' && (  
2   <button  
3     onClick={() => setShowForm(!showForm)}  
4     className="btn btn-primary"  
5   >  
6     {showForm ? 'Cancel' : '+ Post Project'}  
7   </button>  
8 )}
```

7.5.3 Bids Page

The Bids page shows two completely different interfaces depending on the role. For a client, it shows a two column layout where the left side lists their projects and the right

side shows the bids received on the selected project along with an Accept Bid button for pending bids.

For a freelancer, it shows a list of all bids they have submitted along with their current status, and a form to place a new bid on any open project.

Listing 7.9: Role based rendering in Bids.js

```
1 if (role === 'freelancer') {
2   return (
3     <div className="page">
4       {/* Freelancer view with bid form and bid history */}
5     </div>
6   );
7 }
8
9 return (
10  <div className="page">
11    {/* Client view with project selector and bids panel */}
12  </div>
13 );
```

The Accept Bid button only appears for clients and only on bids that have a status of pending. This means that once a bid has been accepted or rejected, the button disappears from the interface.

Listing 7.10: Accept button only shown for pending bids to clients

```
1 {b[3] === 'pending' && (
2   <button
3     onClick={() => handleAccept(b[0])}
4     className="btn btn-primary"
5   >
6     Accept Bid
7   </button>
8 )}
```

7.5.4 Payments Page

The Payments page shows a different heading and a different form depending on the role. For a client it shows a heading of Payments and includes the Record Payment form. For a freelancer it shows a heading of Earnings and hides the form entirely since freelancers only receive payments, they do not record them.

Listing 7.11: Role based heading and form in Payments.js

```
1 <h2 style={{ fontSize: '28px', fontWeight: 700 }}>
2   {role === 'client' ? 'Payments' : 'Earnings'}
3 </h2>
4
5 {role === 'client' && contracts.length > 0 && (
6   <button
7     onClick={() => setShowForm(!showForm)}
8     className="btn btn-primary"
9   >
10    {showForm ? 'Cancel' : '+ Record Payment'}
11  </button>
12 )}
```

7.6 Backend Enforcement of Role Based Access

It is important to note that the frontend role checks are only for the user interface. The real enforcement of access control happens on the backend. Every protected route on the backend verifies the JWT token before processing any request. The token contains the role, and the backend uses it to decide what data to return or whether to allow an operation.

For example, when a POST request is made to create a project, the backend checks that the role in the token is client. If a freelancer somehow tried to send this request directly, the backend would reject it with a 403 Forbidden error.

Listing 7.12: Backend role check for project creation

```
1 if (req.user.role !== 'client')
2   return res.status(403).json({
3     error: 'Only clients can post projects'
4   });
```

Similarly the admin routes on the backend have a separate middleware function that checks for the admin role before allowing access to any of the admin data endpoints.

Listing 7.13: Admin only middleware in backend admin.js

```
1 function adminOnly(req, res, next) {
2   if (req.user.role !== 'admin')
3     return res.status(403).json({
4       error: 'Admin access only'
5     });
6 }
```

```

6   next();
7 }

```

This means that even if someone bypassed the frontend completely and sent a direct HTTP request to an admin endpoint, they would be rejected unless their token contained the admin role. And that token can only be issued by the backend after successfully verifying credentials against the admins table in the Oracle database.

7.7 Summary of Role Based Access

The table below summarizes what each role can see and do in the frontend.

Feature	Client	Freelancer	Admin
Dashboard	Own projects and bids received	Open projects and own bids	All platform stats
Projects	Own projects, post new	Browse all open	View all
Bids	Bids received, accept	Submit bids	View all
Contracts	Own contracts	Own contracts	View all
Payments	Record payment	View earnings	View all
Reviews	Leave and receive	Leave and receive	View all
Disputes	Raise and resolve	Raise and resolve	View all
Notifications	Own notifications	Own notifications	View all
Skills	Not shown	Not shown	View all

Table 7.1: Role Based Access Summary

7.8 Summary

Role based access control in SkillLink operates at three levels. At the route level, `PrivateRoute` and `AdminRoute` in `App.js` prevent unauthorized access to entire sections of the application. At the navigation level, the `Sidebar` shows different links depending on whether the user is a client or a freelancer. At the page level, individual pages fetch different data, show different forms, and hide or show different buttons based on the role. The role itself always originates from the Oracle database at login time and is carried through the session inside a JWT token that the backend verifies on every single request.

Chapter 8

Frontend to Backend Communication: Full Cycle

8.1 Overview

The previous chapters covered data fetching and data sending as separate topics. This chapter brings both together by walking through one complete action from start to finish. The action chosen is a freelancer placing a bid on a project and a client then accepting that bid. This single sequence touches almost every layer of the system: the React frontend, the Axios request, the Express backend, the Oracle database, and the response that comes back to the screen.

Following this full cycle makes it clear how every click on the screen eventually becomes a SQL query in the database, and how the result of that query comes back and changes what the user sees.

8.2 The Starting Point: Two Users Are Logged In

To understand the full cycle, it helps to picture the exact state of the system at the beginning.

A freelancer named Fatima has logged in. When she logged in, the backend ran a SELECT query on the users table joined with the freelancer_profile table, confirmed her credentials, and issued a JWT token with her user ID, email, and role set to freelancer. That token is now stored in her browser's local storage and is attached to every request she makes.

A client named Ali has also logged in in a separate browser. His token contains his user ID, email, and role set to client.

Both tokens came from the Oracle database. The roles in those tokens were determined by which profile table had a matching record for each user.

8.3 Step 1: Freelancer Opens the Projects Page

Fatima clicks Browse Projects in her sidebar. The Projects page component mounts and its `useEffect` hook runs immediately.

Listing 8.1: Projects page fetch on mount in Projects.js

```
1 useEffect(() => { fetchProjects(); }, []);
2
3 const fetchProjects = async () => {
4   try {
5     setLoading(true);
6     const url = role === 'client'
7       ? `${API}/projects/client/${user.user_id}`
8       : `${API}/projects`;
9     const res = await axios.get(
10      url,
11      role === 'client' ? { headers } : {}
12    );
13    setProjects(res.data);
14  } catch (err) {
15    console.error(err);
16  } finally {
17    setLoading(false);
18  }
19 };
```

Because Fatima's role is freelancer, the URL used is the general projects endpoint without any user ID filter. Axios sends a GET request to the backend.

The backend receives the request and runs this query:

Listing 8.2: Backend query for all projects

```
1 SELECT p.project_id, p.title, p.description,
2       p.budget, p.deadline, p.status,
3       u.name as client_name,
4       c.name as category,
5       u.user_id as client_id,
6       (SELECT COUNT(*) FROM bids b
7        WHERE b.project_id = p.project_id) as bid_count
8 FROM projects p
9 JOIN users u ON p.client_id = u.user_id
10 LEFT JOIN project_category pc
```

```
11   ON p.project_id = pc.project_id
12 LEFT JOIN categories c
13   ON pc.category_id = c.category_id
14 ORDER BY p.created_at DESC
```

This query joins four tables: projects, users, project_category, and categories. It also uses a subquery to count the total bids on each project. The result is a list of all projects on the platform with their details.

The backend sends this list back to the frontend as a JSON array. Axios receives the response and the setProjects function stores the array in state. React re-renders the page and each project appears as a card on the screen. The loading message disappears and Fatima can now see all available projects.

8.4 Step 2: Freelancer Places a Bid

Fatima sees a project she likes. She clicks the Place a Bid button. A form appears. She selects the project from a dropdown, enters her bid amount of Rs. 15000, and enters a delivery time of 10 days. She clicks Submit Bid.

The handleSubmitBid function runs in Bids.js:

Listing 8.3: Bid submission handler in Bids.js

```
1 const handleSubmitBid = async () => {
2   if (!bidProject || !amount || !deliveryTime) {
3     setError('All fields are required');
4     return;
5   }
6   setError('');
7   setSubmitting(true);
8   try {
9     await axios.post(
10      `${API}/bids`,
11      {
12        project_id: bidProject,
13        amount,
14        delivery_time: deliveryTime,
15      },
16      { headers }
17    );
18    setSuccess('Bid submitted successfully!');
19    setShowForm(false);
```

```
20     setAmount('');
21     setDeliveryTime('');
22     setBidProject('');
23     fetchData();
24     setTimeout(() => setSuccess(''), 3000);
25 } catch (err) {
26     setError(
27         err.response?.data?.error || 'Failed to submit bid'
28     );
29 } finally {
30     setSubmitting(false);
31 }
32 };
```

The frontend first validates that all three fields are filled. Since they are, it clears any previous error, sets the submitting flag to true to disable the button, and sends a POST request to the backend bids route. The request body contains the project ID, the amount, and the delivery time. The Authorization header contains Fatima's JWT token.

The backend receives this request. The first thing it does is verify the JWT token. It extracts Fatima's user ID and role from it. It then runs two validation queries before inserting anything.

The first validation checks that the project is still open:

Listing 8.4: Backend validation: check project status

```
1 SELECT status FROM projects
2 WHERE project_id = :project_id
```

If the status is not open, the backend returns a 400 error and the frontend shows the message to the user. The bid is not inserted.

The second validation checks that Fatima has not already bid on this project:

Listing 8.5: Backend validation: check for duplicate bid

```
1 SELECT bid_id FROM bids
2 WHERE project_id = :project_id
3 AND freelancer_id = :freelancer_id
```

If a row is found, the backend returns a 409 conflict error. The frontend shows You already placed a bid on this project.

If both validations pass, the backend inserts the new bid:

Listing 8.6: Backend INSERT for new bid

```
1 INSERT INTO bids
2 VALUES (
3     seq_bids.nextval,
4     :project_id,
5     :freelancer_id,
6     :amount,
7     :delivery_time,
8     'pending',
9     sysdate
10 )
```

The freelancer ID used here comes from the JWT token, not from the frontend. This is the security measure that ensures a freelancer can only bid under their own identity.

The backend commits the insert and sends back a 201 Created response with the message Bid submitted successfully. The frontend receives this, shows the success message in green, hides the form, resets all the form fields, and calls `fetchData` to refresh the bids list. Fatima can now see her new bid in the list with a status of pending.

8.5 Step 3: Client Opens the Bids Page

Meanwhile Ali opens his Bids Received page. The Bids page component mounts and `fetchData` runs. Because Ali's role is client, the frontend fetches his projects first:

Listing 8.7: Client bids page initial fetch in Bids.js

```
1 const projRes = await axios.get(
2     `${API}/projects/client/${user.user_id}`,
3     { headers }
4 );
5 setProjects(projRes.data);
6 if (projRes.data.length > 0) {
7     setSelected(projRes.data[0][0]);
8     const bidRes = await axios.get(
9         `${API}/bids/project/${projRes.data[0][0]}`,
10        { headers }
11    );
12    setBids(bidRes.data);
13 }
```

The backend responds to the bids request by running this query:

Listing 8.8: Backend query for bids on a project

```
1 SELECT b.bid_id, b.amount, b.delivery_time,
2       b.status, b.submitted_at,
3       u.name as freelancer_name,
4       u.user_id as freelancer_id,
5       f.rating, f.experience_level, f.bio
6 FROM bids b
7 JOIN users u ON b.freelancer_id = u.user_id
8 JOIN freelancer_profile f
9   ON u.user_id = f.freelancer_id
10 WHERE b.project_id = :project_id
11 ORDER BY b.amount ASC
```

This query joins three tables: bids, users, and freelancer_profile. It gives Ali not just the bid amount and delivery time but also the freelancer's name, rating, and experience level so he has everything he needs to evaluate each bid. The results are ordered by amount ascending so the lowest bid appears first.

The frontend receives this data and renders each bid as a card. Fatima's bid appears in the list showing her name, her bid of Rs. 15000, 10 days delivery, and a pending status badge. An Accept Bid button appears next to it because the status is pending.

8.6 Step 4: Client Accepts the Bid

Ali decides to accept Fatima's bid. He clicks the Accept Bid button. A confirmation dialog appears asking Are you sure? A contract will be created automatically. He clicks OK.

The handleAccept function runs:

Listing 8.9: Bid acceptance handler in Bids.js

```
1 const handleAccept = async (bidId) => {
2   if (!window.confirm(
3     'Accept this bid? A contract will be created automatically.'
4   )) return;
5   try {
6     await axios.put(
7       `${API}/bids/${bidId}/accept`,
8       {},
9       { headers }
10    );
11    setSuccess(
```

```
12     'Bid accepted! Contract created successfully.'
13   );
14   fetchBidsForProject(selected);
15   setTimeout(() => setSuccess(''), 4000);
16 } catch (err) {
17   setError(
18     err.response?.data?.error || 'Failed to accept bid'
19   );
20   setTimeout(() => setError(''), 4000);
21 }
22 };
```

The frontend sends a PUT request to the backend with the bid ID in the URL. The request body is empty because the backend already has everything it needs from the bid ID and the JWT token.

The backend receives this request and begins a database transaction. It does not commit anything until all four operations are complete.

The first operation reads the bid details:

Listing 8.10: Backend: read bid details before accepting

```
1 SELECT bid_id, project_id, freelancer_id,
2        amount, delivery_time
3 FROM bids
4 WHERE bid_id = :bid_id
```

The second operation marks this bid as accepted:

Listing 8.11: Backend: UPDATE bid status to accepted

```
1 UPDATE bids
2 SET status = 'accepted'
3 WHERE bid_id = :bid_id
```

The third operation rejects all other bids on the same project:

Listing 8.12: Backend: UPDATE all other bids to rejected

```
1 UPDATE bids
2 SET status = 'rejected'
3 WHERE project_id = :project_id
4 AND bid_id != :bid_id
```

The fourth operation updates the project status:

Listing 8.13: Backend: UPDATE project status to in_progress

```
1 UPDATE projects
2 SET status = 'in_progress'
3 WHERE project_id = :project_id
```

The fifth operation creates the contract automatically. The end date is calculated by adding the delivery time in days to the current date:

Listing 8.14: Backend: INSERT new contract

```
1 INSERT INTO contracts
2 VALUES (
3     seq_contracts.nextval,
4     :bid_id,
5     :amount,
6     sysdate,
7     sysdate + :delivery_time,
8     'active'
9 )
```

All five operations are committed together in a single transaction. If any one of them fails, all of them are rolled back and the database remains unchanged. This protects the integrity of the data.

The backend sends back a 200 OK response with the message Bid accepted and contract created successfully. The frontend receives this, shows the success message, and calls `fetchBidsForProject` to refresh the bids list. Ali can now see that Fatima's bid shows an accepted badge and all other bids show a rejected badge. The Accept Bid button has disappeared because the bid status is no longer pending.

8.7 Step 5: The Contract Appears

Both Ali and Fatima can now go to their Contracts page and see the new contract. When either of them opens the Contracts page, the frontend sends a GET request to the backend:

Listing 8.15: Contracts fetch in Contracts.js

```
1 const res = await axios.get(
2     `${API}/contracts/my`,
3     { headers }
4 );
5 setContracts(res.data);
```

The backend runs this query to find all contracts related to the logged in user:

Listing 8.16: Backend query for user contracts

```
1 SELECT c.contract_id, c.amount, c.start_date,
2       c.end_date, c.status,
3       p.title as project_title,
4       u1.name as client_name,
5       u2.name as freelancer_name,
6       u1.user_id as client_id,
7       u2.user_id as freelancer_id
8 FROM contracts c
9 JOIN bids b ON c.bid_id = b.bid_id
10 JOIN projects p ON b.project_id = p.project_id
11 JOIN users u1 ON p.client_id = u1.user_id
12 JOIN users u2 ON b.freelancer_id = u2.user_id
13 WHERE u1.user_id = :user_id
14 OR u2.user_id = :user_id
15 ORDER BY c.start_date DESC
```

This query joins five tables: contracts, bids, projects, and two instances of the users table, one for the client and one for the freelancer. The WHERE clause uses the user ID from the JWT token and returns the contract regardless of whether the logged in user is the client or the freelancer on that contract.

The frontend receives the contract data and renders it as a card showing the project title, the contract amount, the start and end dates, the status of active, and the name of the other party. For Ali it shows the freelancer name. For Fatima it shows the client name.

8.8 The Complete Cycle Summarized

The table below shows every step of the full cycle from the first click to the final screen update, including the component involved, the HTTP request made, and the database operation that resulted.

8.9 Summary

This chapter followed a single user journey from a freelancer browsing projects all the way through to both parties seeing a new contract on their screens. Every step in this journey involved a React component sending an HTTP request through Axios to the backend, the backend running one or more SQL queries against the Oracle database, and the result coming back to update the screen. The most complex step was the bid acceptance, which

Step	Component	HTTP	DB Op	Tables Touched
1	Projects.js	GET /projects	SELECT	projects, users, categories, bids
2	Bids.js	POST /bids	INSERT	bids
3	Bids.js	GET /bids/project/:id	SELECT	bids, users, freelancer_profile
4a	Bids.js	PUT /bids/:id/accept	UPDATE	bids (accepted)
4b	Bids.js	PUT /bids/:id/accept	UPDATE	bids (others rejected)
4c	Bids.js	PUT /bids/:id/accept	UPDATE	projects
4d	Bids.js	PUT /bids/:id/accept	INSERT	contracts
5	Contracts.js	GET /contracts/my	SELECT	contracts, bids, projects, users

Table 8.1: Complete Frontend to Backend Communication Cycle

triggered five database operations across four tables inside a single transaction. This full cycle demonstrates how the frontend, backend, and database work together as a single connected system.

Chapter 9

Admin Panel

9.1 Overview

The admin panel is a completely separate section of the SkillLink frontend that gives a privileged user a full view of all the data stored in the Oracle database. While regular users only see their own data filtered by their user ID and role, the admin sees everything. Every table in the database has a corresponding admin page that displays its contents in a structured table with search and filter controls.

The admin panel is accessible through a separate login page at the path `/admin/login` and is protected by the `AdminRoute` component in `App.js` which checks that the logged in user has the role of admin before allowing access. The admin role is issued only after verifying credentials against the `admins` table in the Oracle database, which is completely separate from the regular users table.

This chapter goes through each admin page, explains what data it shows, which database tables it queries, and what actions the admin can perform.

9.2 Admin Authentication

Before the admin can access any data page, they must log in through `AdminLogin.js`. This page sends a POST request to a separate backend endpoint that queries the `admins` table rather than the `users` table.

Listing 9.1: Admin login request in `AdminLogin.js`

```
1 const res = await axios.post(  
2   'http://localhost:5000/api/admin/auth/login',  
3   { email, password }  
4 );  
5 login(  
6   {  
7     user_id: res.data.admin_id,
```

```
8     name:      res.data.name ,
9     email:     res.data.email ,
10    role:      'admin' ,
11  },
12  res.data.token
13 );
14 window.location.href = '/admin';
```

The backend checks the admins table and if the credentials match, issues a JWT token with the role set to admin. The frontend saves this token to local storage using the same AuthContext login function used by regular users. The admin is then redirected to the admin dashboard.

Every subsequent request from the admin panel includes this token in the Authorization header. The backend verifies it and checks that the role is admin before returning any data. If a regular user somehow navigated to an admin route, the AdminRoute component would redirect them to /admin/login, and even if they sent a direct HTTP request to an admin endpoint, the backend would reject it with a 403 Forbidden error.

9.3 Admin Layout and Navigation

Once logged in, the admin sees a layout that consists of the AdminSidebar on the left and the current admin page on the right. The AdminSidebar uses an orange color theme to visually distinguish it from the regular user sidebar which uses teal. This makes it immediately clear when someone is in the admin area.

The AdminSidebar contains links to all ten data pages plus the dashboard:

Listing 9.2: Admin navigation links in AdminSidebar.js

```
1  const links = [
2    { to: '/admin',          label: 'Dashboard'    },
3    { to: '/admin/users',    label: 'Users'      },
4    { to: '/admin/projects', label: 'Projects'   },
5    { to: '/admin/bids',     label: 'Bids'       },
6    { to: '/admin/contracts', label: 'Contracts'  },
7    { to: '/admin/payments', label: 'Payments'   },
8    { to: '/admin/reviews',  label: 'Reviews'    },
9    { to: '/admin/skills',   label: 'Skills'     },
10   { to: '/admin/disputes',  label: 'Disputes'   },
11   { to: '/admin/notifications', label: 'Notifications' },
12 ];
```


Each link corresponds to one admin page. Each admin page follows the same general structure: a header with a title and record count, a search or filter bar, and a table showing all records from the relevant database tables.

9.4 Admin Dashboard

The admin dashboard is the first page the admin sees after logging in. It shows a grid of stat cards, one for each major table in the database, along with a count of how many records are in that table. It also shows recent activity including the latest projects, bids, and contracts.

Listing 9.3: Admin dashboard data fetch in AdminDashboard.js

```
1 const [
2   usersRes, projectsRes, bidsRes,
3   contractsRes, paymentsRes, reviewsRes,
4   disputesRes, notifRes, skillsRes
5 ] = await Promise.all([
6   axios.get(`${API}/users`, { headers }),
7   axios.get(`${API}/projects`, { headers }),
8   axios.get(`${API}/bids`, { headers }),
9   axios.get(`${API}/contracts`, { headers }),
10  axios.get(`${API}/payments`, { headers }),
11  axios.get(`${API}/reviews`, { headers }),
12  axios.get(`${API}/disputes`, { headers }),
13  axios.get(`${API}/notifications`, { headers }),
14  axios.get(`${API}/skills`, { headers }),
15 ]);
```

All nine requests are sent in parallel using Promise.all. The backend responds to each one with a full SELECT from the relevant table joined with whatever other tables are needed to give meaningful data. The frontend uses only the length of each result array to display the count on each stat card.

The stat cards show the total count of users, projects, bids, contracts, payments, reviews, disputes, notifications, and skills. Below the stat cards, the dashboard shows the five most recent projects and the five most recent bids as quick reference panels.

The database tables involved in the admin dashboard are all fifteen tables in the Oracle database, queried through nine parallel GET requests.

9.5 Admin Users Page

The admin users page shows a complete list of all registered users on the platform. It joins the users table with both the client_profile and freelancer_profile tables so that the admin can see the role of each user alongside their basic information.

Listing 9.4: Admin users fetch in AdminUsers.js

```
1 const res = await axios.get(  
2   '${API}/users',  
3   { headers }  
4 );  
5 setUsers(res.data);
```

The backend responds with a query that joins three tables:

Listing 9.5: Backend query for admin users view

```
1 SELECT u.user_id, u.name, u.email, u.phone,  
2        u.created_at,  
3        c.company_name, c.location,  
4        f.hourly_rate, f.rating,  
5        f.experience_level, f.availability,  
6        CASE  
7          WHEN c.client_id IS NOT NULL  
8            THEN 'client'  
9          WHEN f.freelancer_id IS NOT NULL  
10           THEN 'freelancer'  
11        END as role  
12 FROM users u  
13 LEFT JOIN client_profile c  
14   ON u.user_id = c.client_id  
15 LEFT JOIN freelancer_profile f  
16   ON u.user_id = f.freelancer_id  
17 ORDER BY u.created_at DESC
```

The admin page displays each user in a table row showing their name, email, phone, role badge, and registration date. For clients it also shows their company name and location. For freelancers it shows their hourly rate, rating, and experience level.

The page includes a search bar that filters users by name or email on the frontend without making a new backend request. There is also a role filter with buttons for All, Client, and Freelancer that filters the displayed list instantly.

The database tables involved are users, client_profile, and freelancer_profile.

9.6 Admin Projects Page

The admin projects page shows all projects on the platform regardless of which client posted them. It joins the projects table with the users table to show the client name and with the categories table to show the project category.

Listing 9.6: Admin projects fetch in AdminProjects.js

```
1 const res = await axios.get(  
2   `${API}/projects`,  
3   { headers }  
4 );  
5 setProjects(res.data);
```

Each project row in the table shows the project ID, title, client name, budget, deadline, status badge, category, and bid count. The status badge uses color coding: open projects show in green, in progress projects show in blue, and completed projects show in purple.

The page includes a search bar for filtering by title or client name and status filter buttons for All, Open, In Progress, and Completed.

The database tables involved are projects, users, project_category, categories, and bids.

9.7 Admin Bids Page

The admin bids page shows all bids placed on the platform. It joins the bids table with the users table to show the freelancer name and with the projects table to show the project title.

Listing 9.7: Admin bids fetch in AdminBids.js

```
1 const res = await axios.get(  
2   `${API}/bids`,  
3   { headers }  
4 );  
5 setBids(res.data);
```

Each bid row shows the bid ID, freelancer name, project title, bid amount, delivery time in days, status badge, and submission date. The status badge shows pending in yellow, accepted in green, and rejected in red.

The page includes a search bar and status filter buttons. The admin can see at a glance how many bids are in each state across the entire platform.

The database tables involved are bids, users, and projects.

9.8 Admin Contracts Page

The admin contracts page shows all contracts on the platform. It performs the most complex join of any admin page because a contract connects a bid to a project to two users.

Listing 9.8: Admin contracts fetch in AdminContracts.js

```
1 const res = await axios.get(  
2   `${API}/contracts`,  
3   { headers }  
4 );  
5 setContracts(res.data);
```

The backend responds with a query that joins five tables:

Listing 9.9: Backend query for admin contracts view

```
1 SELECT c.contract_id, c.amount,  
2        c.start_date, c.end_date, c.status,  
3        p.title as project_title,  
4        u1.name as client_name,  
5        u2.name as freelancer_name,  
6        u1.user_id as client_id,  
7        u2.user_id as freelancer_id  
8 FROM contracts c  
9 JOIN bids b ON c.bid_id = b.bid_id  
10 JOIN projects p ON b.project_id = p.project_id  
11 JOIN users u1 ON p.client_id = u1.user_id  
12 JOIN users u2 ON b.freelancer_id = u2.user_id  
13 ORDER BY c.start_date DESC
```

Each contract row shows the contract ID, project title, client name, freelancer name, contract amount, start date, end date, and status badge. The status shows active in blue and completed in green.

The database tables involved are contracts, bids, projects, and users joined twice.

9.9 Admin Payments Page

The admin payments page shows all payment transactions recorded on the platform. It joins the payments table through to the contracts, bids, projects, and users tables to give the full context of each payment.

Listing 9.10: Admin payments fetch in AdminPayments.js

```
1 const res = await axios.get(  
2   `${API}/payments`,  
3   { headers }  
4 );  
5 setPayments(res.data);
```

Each payment row shows the payment ID, project title, client name, freelancer name, amount, payment method, payment date, and status badge. The page also shows a summary card at the top displaying the total amount of all payments combined, giving the admin a quick view of the total money that has moved through the platform.

The database tables involved are payments, contracts, bids, projects, and users.

9.10 Admin Reviews Page

The admin reviews page shows all reviews submitted on the platform. It joins the reviews table with the users table twice, once for the reviewer and once for the person being reviewed.

Listing 9.11: Admin reviews fetch in AdminReviews.js

```
1 const res = await axios.get(  
2   `${API}/reviews`,  
3   { headers }  
4 );  
5 setReviews(res.data);
```

Each review row shows the review ID, reviewer name, reviewee name, star rating displayed as filled and empty stars, the comment if one was left, and the date. The admin can see all reviews across the platform and identify any patterns in feedback.

The database tables involved are reviews and users joined twice.

9.11 Admin Skills Page

The admin skills page is unique among the admin pages because it shows data from the skills system, which is not directly visible to regular users. It displays all skills registered in the platform and which freelancers have each skill.

Listing 9.12: Admin skills fetch in AdminSkills.js

```
1 const res = await axios.get(  
2   `${API}/skills`,  
3   { headers }  
4 );  
5 setSkills(res.data);
```

```
2   `${API}/skills`,
3   { headers }
4 );
5 setSkills(res.data);
```

The backend responds with a query that joins the skills table with the user_skills table and the users table to show which freelancer has each skill:

Listing 9.13: Backend query for admin skills view

```
1 SELECT s.skill_id, s.name, s.category,
2        u.name as freelancer_name,
3        u.user_id as freelancer_id,
4        f.experience_level
5 FROM skills s
6 LEFT JOIN user_skills us
7     ON s.skill_id = us.skill_id
8 LEFT JOIN users u
9     ON us.user_id = u.user_id
10 LEFT JOIN freelancer_profile f
11     ON u.user_id = f.freelancer_id
12 ORDER BY s.category, s.name
```

Each row shows the skill name, category, and the freelancer who has that skill along with their experience level. Skills that no freelancer has claimed yet are also shown with empty freelancer fields.

The database tables involved are skills, user_skills, users, and freelancer_profile.

9.12 Admin Disputes Page

The admin disputes page shows all disputes raised on the platform. It joins the disputes table with the users table to show who raised the dispute, with the contracts table to show the contract amount, and with the projects table to show the project title.

Listing 9.14: Admin disputes fetch in AdminDisputes.js

```
1 const res = await axios.get(
2   `${API}/disputes`,
3   { headers }
4 );
5 setDisputes(res.data);
```

Each dispute row shows the dispute ID, project title, the name of the person who raised the dispute, the contract amount, the reason text, the status badge, and the date raised. Open disputes show in red and resolved disputes show in green.

The admin can mark any open dispute as resolved directly from this page by clicking a Resolve button, which sends a PUT request to the backend:

Listing 9.15: Admin resolve dispute in AdminDisputes.js

```
1 const handleResolve = async (id) => {  
2   await axios.put(  
3     `${API}/disputes/${id}/resolve`,  
4     {},  
5     { headers }  
6   );  
7   fetchData();  
8 };
```

The backend runs an UPDATE on the disputes table changing the status from open to resolved. The page refreshes to show the updated status.

The database tables involved are disputes, users, contracts, bids, and projects.

9.13 Admin Notifications Page

The admin notifications page shows all notifications sent to all users on the platform. It joins the notifications table with the users table to show the name of the user each notification was sent to.

Listing 9.16: Admin notifications fetch in AdminNotifications.js

```
1 const res = await axios.get(  
2   `${API}/notifications`,  
3   { headers }  
4 );  
5 setNotifications(res.data);
```

Each notification row shows the notification ID, the recipient user name, the message text, the notification type badge, the date sent, and whether it has been read or not. The admin can see all notifications across all users, which gives a complete picture of all system activity.

The admin can also create a new notification for any user directly from this page by filling in a form with the user ID, message, and type, and sending a POST request to the backend.

The database tables involved are notifications and users.

9.14 Summary of Admin Pages

The table below summarizes all admin pages, the HTTP requests they make, the database tables they query, and whether they support any write operations.

Admin Page	HTTP	Database Tables	Write Operations
AdminDashboard	GET x9	All 15 tables	None
AdminUsers	GET	users, client_profile, freelancer_profile	None
AdminProjects	GET	projects, users, categories, bids	None
AdminBids	GET	bids, users, projects	None
AdminContracts	GET	contracts, bids, projects, users	None
AdminPayments	GET	payments, contracts, bids, projects, users	None
AdminReviews	GET	reviews, users	None
AdminSkills	GET	skills, user_skills, users, freelancer_profile	None
AdminDisputes	GET, PUT	disputes, users, contracts, projects	Resolve dispute
AdminNotifications	GET, POST	notifications, users	Create notification

Table 9.1: Summary of Admin Pages and Database Operations

9.15 Summary

The admin panel gives a single privileged user a complete view of all data stored across all fifteen tables in the Oracle database. It is protected at two levels: the AdminRoute component on the frontend prevents unauthorized navigation, and the backend middleware rejects any request whose JWT token does not carry the admin role. Each admin page performs a SELECT query that joins multiple tables to give meaningful context to each record. The only admin pages with write operations are AdminDisputes, which can resolve disputes, and AdminNotifications, which can create new notifications. All other admin pages are read only views into the database.

Chapter 10

Error Handling and Validation

10.1 Overview

A well built frontend does not just show data and send requests. It also handles the cases where things go wrong. In SkillLink, error handling and validation happen at two levels. The first level is on the frontend itself, where the code checks user input before sending anything to the backend. The second level is on the backend, where the database operation may fail or be rejected, and the error is sent back to the frontend which then displays it clearly to the user.

This chapter covers both levels and explains exactly how SkillLink handles errors, validates input, manages loading states, and displays success and failure messages to the user.

10.2 Frontend Input Validation

Frontend validation is the first line of defense. Before any Axios request is sent, the handler function checks that the required fields are filled in and that the values make sense. If a check fails, the function sets an error message in state and returns early without making any network request. This saves time and avoids unnecessary backend calls.

The pattern used across all forms in SkillLink is consistent. Required fields are checked first. If any are empty, the error state is set and the function exits. Only if all checks pass does the Axios request get sent.

Listing 10.1: Frontend validation before sending a request

```
1 const handleSubmitBid = async () => {  
2   if (!bidProject || !amount || !deliveryTime) {  
3     setError('All fields are required');  
4     return;  
5   }
```

```
6   setError('');
7   setSubmitting(true);
8   try {
9     await axios.post(`${API}/bids`, {
10       project_id:    bidProject,
11       amount,
12       delivery_time: deliveryTime,
13     }, { headers });
14     setSuccess('Bid submitted successfully!');
15   } catch (err) {
16     setError(
17       err.response?.data?.error || 'Failed to submit bid'
18     );
19   } finally {
20     setSubmitting(false);
21   }
22 };
```

In this example from Bids.js, the handler checks three fields before sending. If any one of them is empty or falsy, the error message is set and the function returns immediately. The Axios POST request only runs if all three fields have values.

The table below shows the validation checks applied on each form across the frontend.

Form	Frontend Validation Checks
Login	Email and password must not be empty
Register	Name, email, and password must not be empty
Post Project	Title, budget, and deadline must not be empty
Place a Bid	Project selection, amount, and delivery time must not be empty
Record Payment	Contract selection, amount, and method must not be empty
Submit Review	Contract selection, reviewee, and rating must not be empty
Raise Dispute	Contract selection and reason must not be empty

Table 10.1: Frontend Validation Checks by Form

10.3 Backend Validation and Error Responses

Even after the frontend sends a request, the backend applies its own set of checks before running any database query. These backend checks are more thorough than the frontend checks because the backend has access to the actual database state.

For example, when a freelancer submits a bid, the frontend only checks that the fields are not empty. The backend goes further and checks two additional things that the frontend cannot know without querying the database: whether the project is still open for bids, and whether the freelancer has already placed a bid on that project.

Listing 10.2: Backend validation for bid submission

```
1 // Check project is open
2 const proj = await connection.execute(
3   'SELECT status FROM projects
4   WHERE project_id = :id',
5   { id: project_id }
6 );
7 if (proj.rows[0][0] !== 'open') {
8   return res.status(400).json({
9     error: 'Project is not open for bids'
10  });
11 }
12
13 // Check no duplicate bid
14 const dup = await connection.execute(
15   'SELECT bid_id FROM bids
16   WHERE project_id = :pid
17   AND freelancer_id = :fid',
18   { pid: project_id, fid: freelancer_id }
19 );
20 if (dup.rows.length > 0) {
21   return res.status(409).json({
22     error: 'You already placed a bid on this project'
23   });
24 }
```

When the backend returns an error, it always includes a JSON object with an error field containing a human readable message. The frontend reads this message and displays it directly to the user. This is done using optional chaining to safely access the nested error property:

Listing 10.3: Frontend reading backend error messages

```
1 } catch (err) {
2   setError(
3     err.response?.data?.error || 'Failed to submit bid'
4   );
5 }
```

The optional chaining operator ensures that if the error response does not have the expected shape, for example if the network is completely down and there is no response at all, the fallback message is shown instead of the application crashing.

The table below lists the backend validation checks and the HTTP status codes returned for each error condition.

Condition	Status	Error Message Sent
Email already registered	409	Email already registered
Wrong email or password	401	Invalid email or password
Missing required fields	400	Field name is required
Project not open	400	Project is not open for bids
Duplicate bid	409	You already placed a bid on this project
Contract already completed	400	Contract is already completed
Duplicate review	409	You already reviewed this person for this contract
Dispute already open	409	An open dispute already exists for this contract
Review before completion	400	Can only review after contract is completed
Unauthorized access	403	Admin access only
Record not found	404	Record not found

Table 10.2: Backend Validation Errors and HTTP Status Codes

10.4 Displaying Errors and Success Messages

Every form in SkillLink uses two state variables for user feedback: one for error messages and one for success messages. These are rendered as colored boxes above or below the form depending on which one has content.

Listing 10.4: Error and success message rendering

```

1 {success && (
2   <div className="success-box">{success}</div>
3 )}
4 {error && (
5   <div className="error-box">{error}</div>
6 )}
```

The success-box class applies a green background with a green border and green text. The error-box class applies a red background with a red border and red text. Both are defined in the global index.css file so they look consistent across every page.

Success messages are automatically cleared after a few seconds using setTimeout so they do not stay on the screen permanently after the user has already seen them.

Listing 10.5: Auto-clearing success messages with setTimeout

```
1 setSuccess('Bid submitted successfully!');
2 setTimeout(() => setSuccess(''), 3000);
```

Error messages stay on screen until the user makes another attempt or navigates away. This gives them time to read the message and understand what went wrong before trying again.

10.5 Loading States

Every page and every form in SkillLink has a loading state that tells the user that something is happening. Without loading states, the user would click a button and see nothing happen, which would make the application feel broken.

There are two kinds of loading states in SkillLink. The first is the page loading state, which is active while the initial data fetch is running. The second is the form submitting state, which is active while a POST or PUT request is in progress.

10.5.1 Page Loading State

Each page has a loading state variable that starts as true and is set to false when the data fetch completes. While loading is true, the page shows a simple loading message instead of the empty table or list.

Listing 10.6: Page loading state pattern

```
1 const [loading, setLoading] = useState(true);
2
3 const fetchData = async () => {
4   try {
5     setLoading(true);
6     const res = await axios.get(
7       `${API}/endpoint`,
8       { headers }
9     );
10    setData(res.data);
```

```
11   } catch (err) {
12       console.error(err);
13   } finally {
14       setLoading(false);
15   }
16 };
17
18 return (
19   <div className="page">
20     {loading ? (
21       <p style={{ color: 'var(--muted)' }}>
22         Loading...
23       </p>
24     ) : (
25       null /* render actual content here */
26     )}
27   </div>
28 );
```

The finally block ensures that the loading state is always set to false when the fetch completes, regardless of whether it succeeded or failed. This prevents the loading message from getting stuck on screen if an error occurs.

10.5.2 Form Submitting State

Each form has a submitting state variable that is set to true while the POST or PUT request is in progress. While submitting is true, the submit button is disabled and its label changes to show that the action is in progress.

Listing 10.7: Form submitting state disabling the button

```
1 <button
2   onClick={handleSubmitBid}
3   disabled={submitting}
4   className="btn btn-primary"
5   style={{ opacity: submitting ? 0.6 : 1 }}
6 >
7   {submitting ? 'Submitting...' : 'Submit Bid'}
8 </button>
```

Disabling the button while submitting is important because without it a user could click the button multiple times before the first request completes, which would result in duplicate records being inserted into the database. By disabling the button and reducing its

opacity, the frontend makes it clear that the action is already in progress and prevents duplicate submissions.

10.6 Empty State Handling

In addition to loading states, every page also handles the case where the data fetch succeeds but returns no records. This is called the empty state. Without empty state handling, the page would just show a blank area with no explanation, which would be confusing for the user.

In SkillLink, every list and table checks whether the data array is empty after loading completes. If it is empty, a centered card is shown with a message explaining why there is nothing to display and in some cases a button to take the next action.

Listing 10.8: Empty state handling in Contracts.js

```
1 contracts.length === 0 && (  
2   <div className="card"  
3     style={{ textAlign: 'center', padding: '48px' }}>  
4     <p style={{  
5       color: 'var(--muted)',  
6       fontSize: '14px',  
7       marginBottom: '16px'  
8     }}>  
9       No contracts yet.  
10      {role === 'client'  
11        ? ' Accept a bid to create your first contract.'  
12        : ' Win a bid to get your first contract.'}  
13    </p>  
14    <button  
15      onClick={() => navigate('/dashboard/bids')}  
16      className="btn btn-primary"  
17    >  
18      {role === 'client' ? 'View Bids' : 'Browse Projects'}  
19    </button>  
20  </div>  
21 )
```

The empty state message is also role aware. A client and a freelancer who both have no contracts see different messages and different action buttons because the next step is different for each role. This connects the empty state handling back to the role based access control system described in Chapter 7.

10.7 Network Error Handling

All Axios requests in SkillLink are wrapped in try and catch blocks. If the network request fails completely, for example if the backend server is not running or the internet connection is lost, the catch block runs and the error is handled gracefully.

Listing 10.9: Network error handling with try and catch

```
1 try {  
2   const res = await axios.get(  
3     `${API}/projects`,  
4     { headers }  
5   );  
6   setProjects(res.data);  
7 } catch (err) {  
8   console.error(err);  
9   setError('Failed to load projects. Please try again.');
```

For data fetch errors on page load, the error is logged to the console and the loading state is cleared so the page does not get stuck. For form submission errors, the error message is shown to the user so they know their action did not complete.

The optional chaining used when reading the error message handles both cases cleanly. If the backend returned a structured error response, the message from `err.response.data.error` is shown. If the error was a network failure with no response at all, the fallback string is shown instead.

10.8 Authentication Error Handling

A special category of error is the authentication error. This happens when the JWT token stored in local storage has expired or become invalid. In this case the backend returns a 401 Unauthorized response.

The AuthContext loading state handles the case where the application is still checking local storage for a saved token when it first loads. During this time the PrivateRoute and AdminRoute components return null, preventing any flash of the login page before the check completes.

If a token exists in local storage but has expired, the backend will return a 401 error on the first authenticated request. The frontend catches this in the catch block and displays

an appropriate message. In a future version of the application, this could be handled with an Axios interceptor that automatically redirects to the login page whenever a 401 response is received.

10.9 Confirmation Dialogs

For destructive or irreversible actions, SkillLink uses browser confirmation dialogs before sending the request. This gives the user a chance to cancel if they clicked the button by mistake.

Listing 10.10: Confirmation dialog before accepting a bid

```
1 const handleAccept = async (bidId) => {
2   if (!window.confirm(
3     'Accept this bid? A contract will be ' +
4     'created automatically.'
5   )) return;
6
7   try {
8     await axios.put(
9       `${API}/bids/${bidId}/accept`,
10      {},
11      { headers }
12    );
13    setSuccess(
14      'Bid accepted! Contract created successfully.'
15    );
16  } catch (err) {
17    setError(
18      err.response?.data?.error ||
19      'Failed to accept bid'
20    );
21  }
22 };
```

Confirmation dialogs are used before accepting a bid, before resolving a dispute, and before any other action that cannot be easily undone. Accepting a bid in particular triggers four database operations across three tables and creates a contract, so it is especially important to confirm before proceeding.

10.10 Summary of Error Handling Patterns

The table below summarizes the error handling patterns used across the SkillLink frontend.

Pattern	Description
Frontend validation	Required fields checked before any request is sent
Backend error display	err.response?.data?.error shown to user in red error box
Fallback error message	Generic message shown if network fails with no response
Page loading state	Loading message shown while data is being fetched
Form submitting state	Button disabled and relabeled while request is in progress
Empty state handling	Friendly message shown when data fetch returns no records
Auto-clear success	Success messages cleared after 3 to 4 seconds with setTimeout
Confirmation dialogs	window.confirm used before irreversible database operations
Finally block	Loading and submitting states always reset even if request fails

Table 10.3: Summary of Error Handling Patterns in SkillLink Frontend

10.11 Summary

Error handling and validation in SkillLink operate at every level of the application. The frontend validates input before sending it, the backend validates business rules before writing to the database, and the frontend displays the result clearly whether it is a success or a failure. Loading states keep the user informed while requests are in progress, empty states give meaningful feedback when there is no data to show, and confirmation dialogs protect against accidental irreversible actions. Together these patterns make the application reliable and easy to use even when things do not go as expected.